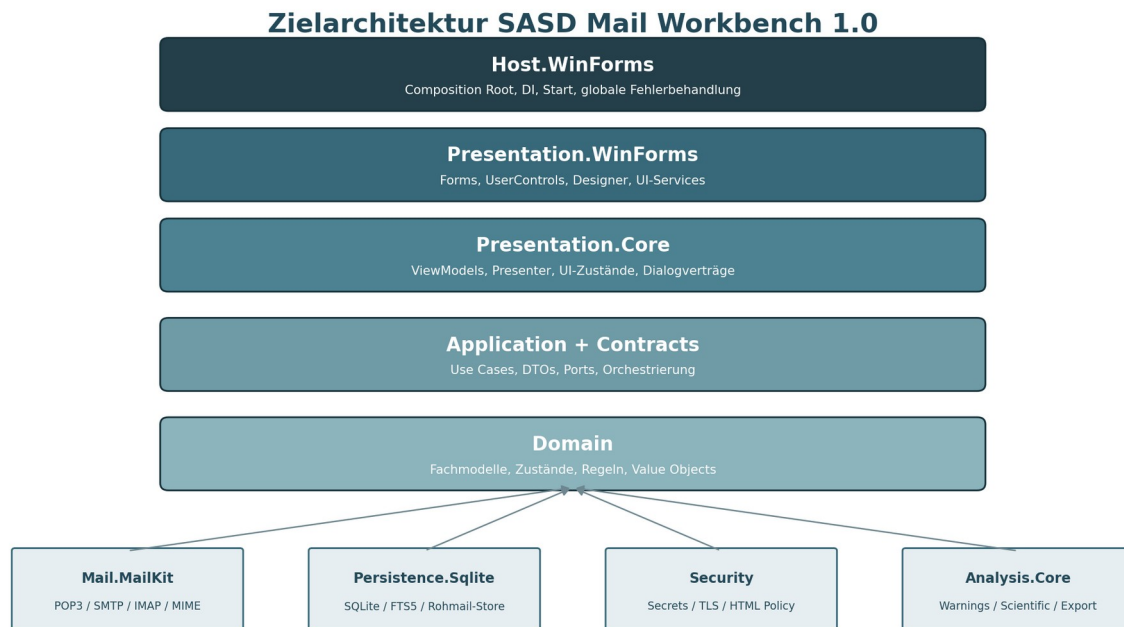


SASD

Mail Workbench

Pflichtenheft Version 1.1

Technische Realisierung des POP3-zentrierten Desktop-Mailclients mit lokalem Warnsystem, Scientific Foundation und austauschbarer Oberfläche



Abhängigkeiten zeigen nach innen; technische Implementierungen werden nur im Host zusammengesetzt.

Abbildung 1: Zielarchitektur der Version 1.0

Auftraggeber / Product Owner: SASD-GmbH

Dokumentversion: 1.0 - vollständiger technischer Entwurf

Zielsoftware: SASD Mail Workbench 1.0.0

Ausgangsbasis: Lastenheft Version 1.0 und Projektstand 0.3.0

Stand: 12. Juli 2026

Status: Zur technischen Prüfung und Freigabe

Dokumentenlenkung

Feld	Festlegung
Dokument	SASD Mail Workbench - Pflichtenheft Version 1.1
Dokumentversion	1.0
Zielrelease	1.0.0
Ausgangsdokument	SASD Mail Workbench - Lastenheft Version 1.0
Implementierter Ausgangsstand	v0.3.0: Domain/Application-Grundgerüst, SQLite-Katalog, Rohmail-Store, Demo und Testprojekt
Technologischer Zielstand	C# 12, .NET 8 für 0.3.1/0.4.0; Migration auf .NET 10 LTS vor Produktrelease; Windows Forms, MailKit/MimeKit, Microsoft.Data.Sqlite
Verbindlichkeit	Dieses Dokument konkretisiert die Realisierung. Abweichungen benötigen ADR, Auswirkungsanalyse und Freigabe.
Anforderungsumfang	131 funktionale und 20 nichtfunktionale Anforderungen sowie 17 Abnahmeszenarien.

Änderungshistorie

Version	Datum	Änderung	Status
0.1	frühere Projektphase	Architektur- und Projektstartentwürfe	historisch
0.3.0	vor diesem Dokument	SQLite-Persistenz statt JSON, lokaler Verarbeitungskern, Testprojekt	implementierter Ausgangsstand
1.0	12.07.2026	Vollständige technische Umsetzung des Lastenhefts; Upgradeziel .NET 10; WinForms, Mailprotokolle, Warnsystem, Scientific Foundation	Entwurf
1.1	12.07.2026	Gestufte Runtime-Strategie und Stabilisierungsaufgaben	Verbindliche Arbeitsgrundlage

Wichtige technische Aktualisierung

Der vorhandene Prototyp zielt auf .NET 8. Für die produktive Version 1.0 wird net10.0-windows festgelegt. .NET 8 befindet sich 2026 bereits in der letzten Supportphase; die Migration wird deshalb als erstes technisches Arbeitspaket durchgeführt. Alle nicht UI-bezogenen Projekte targeten net10.0, das WinForms-Projekt net10.0-windows.

Inhaltsverzeichnis

Kapitel	Inhalt
0	Management Summary
1	Geltungsbereich, Annahmen und Abgrenzung
2	Technologie- und Architekturentscheidungen
3	Solution-, Projekt- und Abhängigkeitsstruktur
4	Laufzeit-, Installations- und Datenverzeichnisstruktur
5	Domänenmodell und Zustandsautomaten
6	Persistenz, Datenbankschema und Migrationen
7	Konten, Providerprofile und Secret-Verwaltung
8	POP3-Abruf und „Get all“
9	Hash-Deduplizierung und Bestandsprüfung
10	MIME-Parsing, HTML-Sicherheit und Darstellung
11	Lokales Warnsystem und Markenprofile
12	Verfassen, Outbox, SMTP und IMAP-Gesendet-Upload
13	Organisation, Suche, Regeln und Anhänge
14	Scientific Foundation und Message Laboratory
15	Extension Model und Plugin-Vorbereitung
16	Presentation Core und WinForms-Oberfläche
17	Logging, Diagnose, Backup und Wiederherstellung
18	Fehlerbehandlung, Wiederanlauf und Parallelität
19	Nichtfunktionale Umsetzung und Qualitätsziele
20	Teststrategie und Abnahme
21	Build, Deployment und Betrieb
22	Umsetzungsplan bis Version 1.0.0

Kapitel	Inhalt
23	Rückverfolgbarkeit aller Anforderungen
A	Datenbankschema
B	Schnittstellenkatalog
C	Glossar und technische Referenzen
D	Freigabe

Hinweis zum Inhaltsverzeichnis

Die Word-Fassung verwendet eine statische, kontrollierte Kapitelübersicht. In Microsoft Word kann zusätzlich über Referenzen > Inhaltsverzeichnis ein dynamisches Verzeichnis aus den Überschriften eingefügt oder aktualisiert werden.

0. Management Summary

Dieses Pflichtenheft beschreibt die konkrete technische Realisierung des fachlich freigegebenen Lastenhefts für SASD Mail Workbench 1.0. Es ist zugleich Technical Design Document, Implementierungsrahmen und Rückverfolgbarkeitsbasis für Entwicklung, Test und Abnahme.

Der Client wird als Windows-Desktopanwendung mit austauschbarer WinForms-Oberfläche implementiert. Der fachliche Kern ist strikt von MailKit, SQLite und WinForms getrennt. Die erste Version verwendet POP3 für den Eingang, SMTP für den Versand und eine eng begrenzte IMAP-Funktion zum Hochladen versendeter Nachrichten in den serverseitigen Gesendet-Ordner.

Die besondere Vollständigkeitsfunktion „Alle Mails abrufen“ lädt jeden noch sichtbaren POP3-Eintrag erneut in einen temporären Stream, bildet einen SHA-256-Hash über die unveränderten Originalbytes und übernimmt nur bislang unbekannte Rohmails. Exakte Duplikate werden bereits vor der kanonischen Speicherung verworfen; bestehende lokale Originale und Servermails bleiben unangetastet.

Das lokale Warnsystem analysiert Header, Absenderidentitäten, Linktexte, tatsächliche Ziele, IDN/Punycode, Authentication-Results, Markenprofile und Anhänge. Es erzeugt erklärbare Einzelbefunde und Warnstufen, ohne eine nicht belegbare Sicherheitsgarantie auszugeben oder Mailinhalte standardmäßig an externe Dienste zu übertragen.

Die Scientific Foundation bewahrt Provenienz, Rohdatenhashes, Analyzer-Versionen und versionierte Analyseläufe. Das Message Laboratory zeigt Nachricht, Analyse, Rohdaten und Historie. Dadurch werden technische Untersuchungen reproduzierbar und später um Datensätze, Statistik und Berichte erweiterbar.

Abnahmeprinzip

Version 1.0 ist nur abnahmefähig, wenn jede MUSS-Anforderung des Lastenhefts einer konkreten Komponente, einem gespeicherten Zustand und mindestens einem automatisierten oder dokumentierten Prüfverfahren zugeordnet ist. Diese Zuordnung befindet sich in Kapitel 23.

1. Geltungsbereich, Annahmen und Abgrenzung

1.1 Geltungsbereich

- Lokale Windows-Desktopanwendung für einen angemeldeten Benutzer und mehrere Mailkonten.
- POP3S für den Nachrichteneingang; SMTP mit TLS/STARTTLS für Versand; IMAPS ausschließlich für Ordnerermittlung und APPEND nach Gesendet.
- Lokale SQLite-Datenbank, lokale unveränderte EML-Rohdateien und geschützte Betriebssystemablage für Secrets.
- WinForms als erste UI-Technologie; fachliche und Präsentationslogik bleiben wiederverwendbar für spätere WPF- oder MAUI-Adapter.
- Lokale Analyse-, Warn- und Scientific-Funktionen ohne vorausgesetztes Herstellerkonto oder Cloudservice.

1.2 Nicht Bestandteil von 1.0

- Vollständige IMAP-Synchronisation eingehender Ordner, Flags und Löschzustände.
- Exchange, Microsoft Graph, Gmail API, JMAP, CalDAV oder CardDAV.
- Kalender, gemeinsame Postfächer, Delegation und Groupware.
- Dynamisches Laden beliebiger Drittanbieter-DLLs oder ein Plugin-Marktplatz.
- Automatische Online-Reputation, Cloud-KI oder Upload von Mailinhalten und Anhängen.
- Garantierte Erkennung von Phishing, Malware oder legitimer Absenderidentität.

1.3 Annahmen

- Der Mailserver unterstützt standardkonforme POP3-UIDL- und TLS-Funktionen; fehlende UIDL wird als eingeschränkter Betriebsmodus behandelt.
- Ein vollständiger POP3-Abruf kann nur Nachrichten finden, die noch auf dem Server sichtbar sind. Durch andere Clients bereits gelöschte Mails sind nicht rekonstruierbar.
- Der Nutzer besitzt gültige Zugangsdaten und gegebenenfalls ein App-Passwort; Providerfreischaltungen wie bei GMX sind außerhalb des Clients, werden aber diagnostisch erklärt.
- Die Anwendung verarbeitet vertrauliche Inhalte lokal und läuft unter dem Windows-Benutzerkonto des Anwenders.

2. Technologie- und Architekturentscheidungen

Entscheidung	Festlegung	Begründung
Programmiersprache	C# 14	Starke Typisierung, asynchrone APIs, gute Windows- und Testunterstützung.
Runtime	.NET 8 für 0.3.1/0.4.0; .NET 10 LTS vor Version 1.0	Visual Studio 2022 bleibt zunächst nutzbar. Die Migration ist wegen des Supportendes von .NET 8 ein verbindliches Release-Gate.

Entscheidung	Festlegung	Begründung
Desktop UI	Windows Forms auf net10.0-windows	Schneller, stabiler visueller Start; UI bleibt austauschbarer Adapter.
Mailbibliothek	MailKit/MimeKit 4.x, zentral gepinnt	POP3, SMTP, IMAP und MIME in einer gepflegten Bibliothek; alle Typen werden hinter Adaptern gekapselt.
Persistenz	Microsoft.Data.Sqlite 10.x + SQLite FTS5	Lokale, portable, transaktionale Datenbank; FTS5 für Volltextsuche.
Rohdaten	Dateisystembasierter EML-Store	Bytegenaue Originale, streambare Verarbeitung, einfache Integritätsprüfung und Backup.
Dependency Injection	Microsoft.Extensions.DependencyInjection	Klare Composition Root, austauschbare Implementierungen und testbare Use Cases.
Logging	Microsoft.Extensions.Logging mit strukturierten Properties	Korrelation, Maskierung und flexible lokale Provider.
Tests	NUnit + kontrollierte Integrationsadapter + Architekturtests	Gute Visual-Studio-Integration und klare Trennung von Unit-, Integrations- und Abnahmetests.
Paketverwaltung	Central Package Management über Directory.Packages.props	Reproduzierbare, zentral sichtbare Abhängigkeitsversionen.

2.1 Architekturprinzipien

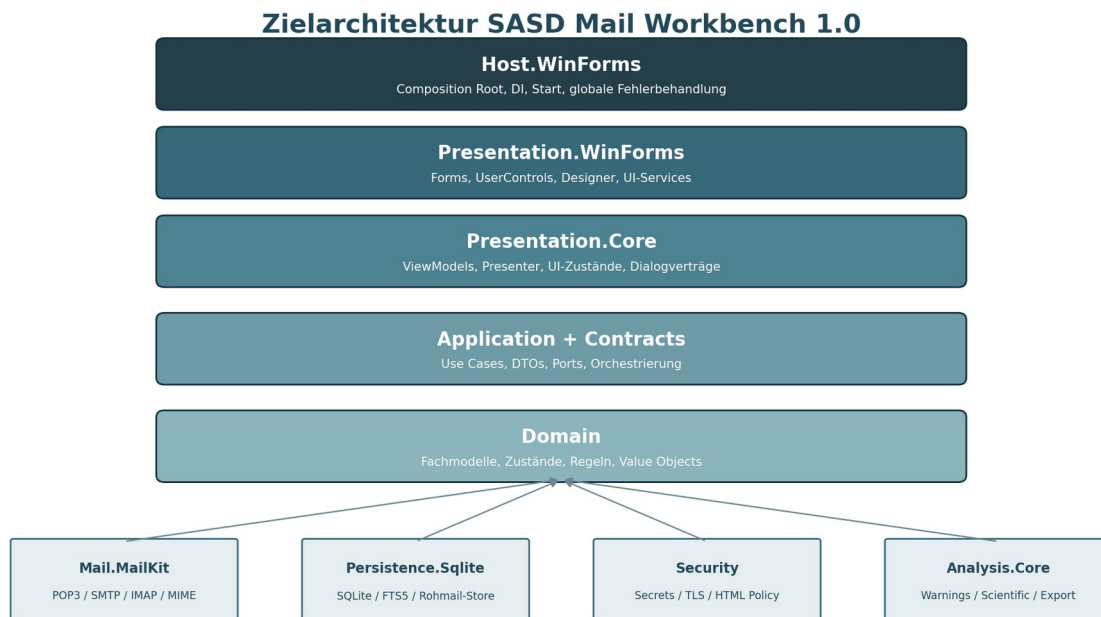
1. Abhängigkeiten zeigen nach innen: Domain kennt keine technischen Frameworks; Application kennt nur Ports und Verträge.
2. MailKit-, SQLite- und WinForms-Typen verlassen ihre jeweiligen Adapterprojekte nicht.
3. Rohmails sind unveränderliche Quelldaten; Parsing, Normalisierung und Analyse erzeugen getrennte, versionierte Ableitungen.
4. Jeder lange oder wiederholbare Ablauf ist abbrechbar, idempotent und besitzt explizite Zustände.
5. Sicherheitsentscheidungen sind lokal, erklärbar und protokollierbar; unsichere Standardwerte sind verboten.
6. Kernmodule verwenden bereits die späteren Extension-Verträge, obwohl Version 1 keine fremden DLLs dynamisch lädt.
7. WinForms enthält nur Darstellung und Interaktion; Eventhandler delegieren sofort an Presentation Core oder Application.

2.2 Architecture Decision Records

ADR	Titel	Status
ADR-001	WinForms als erster austauschbarer UI-Adapter	accepted

ADR	Titel	Status
ADR-002	POP3-zentrierter Eingang mit vollständigem Get-all	accepted
ADR-003	SHA-256-Deduplizierung über unveränderte Rohbytes	accepted
ADR-004	SMTP-Versand plus begrenzter IMAP-APPEND	accepted
ADR-005	Rohdateien im Dateisystem, Metadaten in SQLite	accepted
ADR-006	Lokales erklärbares Warnsystem ohne verpflichtende Clouddienste	accepted
ADR-007	Scientific Foundation mit versionierten Analysis Runs	accepted
ADR-008	Extension Model vorbereiten, dynamische Plugins verschieben	accepted
ADR-009	Gestufte Runtime-Strategie	accepted / Version 1.1

3. Solution-, Projekt- und Abhängigkeitsstruktur



Abhängigkeiten zeigen nach innen; technische Implementierungen werden nur im Host zusammengesetzt.

Abbildung 2: Projekte und Referenzrichtung

```

Sasd.MailWorkbench.sln
├─ src/
│   ├─ Sasd.MailWorkbench.Domain
│   ├─ Sasd.MailWorkbench.Application.Contracts
│   ├─ Sasd.MailWorkbench.Application
│   ├─ Sasd.MailWorkbench.Mail.Abstractions
│   ├─ Sasd.MailWorkbench.Mail.MailKit
│   ├─ Sasd.MailWorkbench.Persistence.Abstractions
│   ├─ Sasd.MailWorkbench.Persistence.Sqlite
│   ├─ Sasd.MailWorkbench.Security
│   ├─ Sasd.MailWorkbench.Analysis.Contracts
│   ├─ Sasd.MailWorkbench.Analysis.Core
│   ├─ Sasd.MailWorkbench.ExtensionModel
│   ├─ Sasd.MailWorkbench.Presentation.Core
│   ├─ Sasd.MailWorkbench.Presentation.WinForms
│   └─ Sasd.MailWorkbench.Host.WinForms
├─ tests/
│   ├─ Sasd.MailWorkbench.Domain.Tests
│   ├─ Sasd.MailWorkbench.Application.Tests
│   ├─ Sasd.MailWorkbench.Mail.Tests
│   ├─ Sasd.MailWorkbench.Persistence.Tests
│   ├─ Sasd.MailWorkbench.Analysis.Tests
│   ├─ Sasd.MailWorkbench.Presentation.Core.Tests
│   ├─ Sasd.MailWorkbench.IntegrationTests
│   ├─ Sasd.MailWorkbench.ArchitectureTests
│   └─ Sasd.MailWorkbench.AcceptanceTests
└─ docs/

```

Projekt	Verantwortung	Zulässige Abhängigkeiten
Domain	Fachmodelle, Value Objects, Zustände, reine Regeln	keine externen NuGet-Pakete außer BCL
Application.Contracts	Requests, Responses, DTOs, Progress- und Fehlerverträge	Domain
Application	Use Cases, Orchestrierung, Ports	Domain, Contracts, Abstractions
Mail.Abstractions	POP3-/SMTP-/IMAP-/MIME-Ports	Domain/Contracts
Mail.MailKit	MailKit-Adapter und Mapping	Mail.Abstractions, MailKit/MimeKit
Persistence.Abstractions	Repositories, Unit of Work, Raw Store	Domain/Contracts
Persistence.Sqlite	SQLite, FTS5, Migrationen, Dateisystemstore	Persistence.Abstractions, Microsoft.Data.Sqlite
Security	Secret Store, HTML-/URL-Policy, Maskierung	Domain/Contracts
Analysis.Contracts	Analyzer-, Finding- und Resultatverträge	Domain, ExtensionModel
Analysis.Core	Warnregeln, Aggregation, Scientific Services	Analysis.Contracts, Security
ExtensionModel	kleine stabile Erweiterungsverträge	BCL, Domain DTOs

Projekt	Verantwortung	Zulässige Abhängigkeiten
Presentation.Core	ViewModels, Presenter, UI-Zustände, Dialogports	Application.Contracts
Presentation.WinForms	Forms, Controls, Ressourcen, UI-Service-Adapter	Presentation.Core, WinForms
Host.WinForms	Composition Root, Konfiguration, Logging, Start	alle konkreten Implementierungen

3.1 Verbotene Referenzen

- Domain, Application, Mail.Abstractions, Persistence.Abstractions, Security, Analysis.Contracts, ExtensionModel und Presentation.Core dürfen kein System.Windows.Forms referenzieren.
- Presentation.WinForms darf MailKit und Microsoft.Data.Sqlite nicht referenzieren.
- Domain darf keine Microsoft.Extensions-, JSON-, Datenbank-, Netzwerk- oder UI-Abhängigkeit besitzen.
- MailKit-Typen dürfen nur im Projekt Mail.MailKit und in dessen internen Tests vorkommen.
- SqlConnection, SqlCommand und SQL-Details dürfen nur im Projekt Persistence.Sqlite vorkommen.
- Host.WinForms ist der einzige Composition Root und darf konkrete Implementierungen zusammenführen.

4. Laufzeit-, Installations- und Datenverzeichnisstruktur

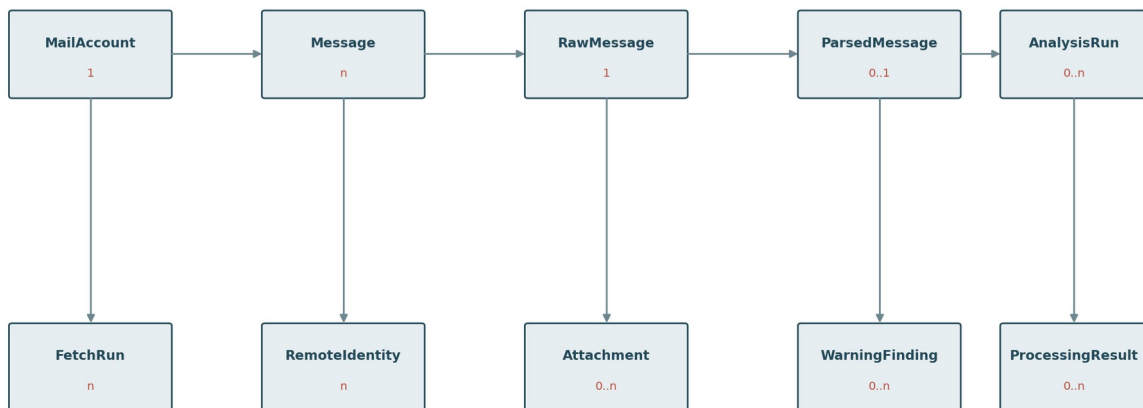
%LocalAppData%\SASD-GmbH\MailWorkbench\	nicht geheime Konfiguration, Providerprofile, Themes
├ config\	
├ database\mailworkbench.db	
├ raw-mails\<account>\<yyyy>\<mm>\<message-id>.eml	
├ attachments-cache\	optional, jederzeit rekonstruierbar
├ drafts\	verwaltete Entwurfsanhänge
├ temp\	laufbezogene temporäre Dateien
├ logs\	strukturierte lokale Logs
├ backups\	vom Nutzer erzeugte Sicherungen
├ exports\	explizite Exporte
├ diagnostics\	vorbereitete Diagnosepakete

Bereich	Regel
Program Files	Nur Binärdateien und unveränderliche Ressourcen; keine Benutzerdaten.
LocalAppData	Profilbezogene Daten; Standardablage für DB, Rohmails, Logs und Cache.
Secrets	Nicht im Dateibaum; Windows Credential Manager/DPAPI hinter ISecretStore.

Bereich	Regel
Temp	Jeder Lauf erhält Unterordner; Startup-Reconciler entfernt nur eindeutig verwaiste Dateien.
Dateiberechtigungen	Vererbung des Benutzerprofils; keine globalen Schreibrechte.
Portable Mode	Nicht Teil von 1.0, da Secret- und Datenschutzmodell separat spezifiziert werden müsste.

5. Domänenmodell und Zustandsautomaten

Kernobjekte der Persistenz



Rohdateien liegen im Dateisystem; SQLite speichert Identität, Status, Indizes, Provenienz und versionierte Ergebnisse.

Abbildung 3: Fachliche Kernobjekte und Kardinalitäten

Typ	Schlüsselattribute	Invarianten
MailAccount	AccountId, DisplayName, EmailAddress, Incoming/Outgoing settings, SecretReference, IsEnabled	Keine Secretwerte; Ports 1..65535; SecurityMode niemals None im Produktivbetrieb.
RawMessage	MessageId, AccountId, Uidl, RawSha256, RawLength, StoragePath, Provenance	Nach RawStored unveränderlich; Hash und Länge entsprechen exakt der Datei.
ParsedMessage	MessageId, ParserVersion, Header/Body DTOs, Attachment descriptors	Nur abgeleitete Daten; kann neu erzeugt werden.
FetchRun	RunId, AccountId, Mode, Started/Finished, counters, state	Zähler monoton; Endstatus genau einmal gesetzt.
OutboxItem	OutboxId, MimePath, MessageIdHeader, SMTP state, attempt counters	Versand bytes bleiben pro Item stabil; Retry ändert nicht Message-ID.

Typ	Schlüsselattribute	Invarianten
SentUploadJob	UploadId, SentMessageId, target folder, IMAP state, UID	SMTP-Erfolg unabhängig vom Uploadstatus.
AnalysisRun	RunId, AnalyzerId/Version, InputHash, ConfigHash, status, Resultjson	Abgeschlossenes Ergebnis unveränderlich; neue Analyse erzeugt neuen Run.
WarningFinding	FindingId, RuleId/Version, Severity, Evidence, Explanation, Recommendation	Keine Sicherheitsgarantie; Evidenz muss menschenlesbar sein.
BrandTrustProfile	ProfileId, Name, domains, providers, version, validity	Lokale Änderung versioniert; keine automatische Vertrauensausweitung.
DatasetDefinition	DatasetId, version, selection criteria, explicit members, manifest	Versioniert und reproduzierbar; Originaldaten werden nicht verändert.

5.1 Zustandsmodelle

Aggregat	Zustände	Zulässige Hauptübergänge
Message ingestion	Discovered, Fetching, TempStored, Duplicate, RawStored, Parsed, Analyzed, Failed, RequiresReview	Discovered→Fetching→TempStored→Duplicate RawStored→Parsed→Analyzed
FetchRun	Planned, Running, Cancelling, Completed, CompletedWithErrors, Cancelled, Failed	Planned→Running; Running→Completed CompletedWithErrors Cancelling Failed
SMTP	Pending, Sending, Sent, SendUncertain, RetryRequired, FailedPermanent	Pending→Sending→Sent SendUncertain RetryRequired FailedPermanent
IMAP Sent upload	Pending, Uploading, Uploaded, UploadUncertain, RetryRequired, FailedPermanent	Pending→Uploading→Uploaded UploadUncertain RetryRequired FailedPermanent
Analysis	Queued, Running, Completed, CompletedWithWarnings, Failed, Superseded	Queued→Running→Completed CompletedWithWarnings Failed; abgeschlossen→Superseded

6. Persistenz, Datenbankschema und Migrationen

SQLite ist Katalog, Zustands- und Suchdatenbank. Rohmails und große Binärdaten werden nicht als primäre BLOBs gespeichert, sondern als unveränderte Dateien referenziert. Fremdschlüssel sind aktiviert; Write-Ahead Logging wird verwendet, sofern die Plattformprüfung keine Inkonsistenz zeigt.

Tabelle	Zweck	Wichtige Indizes
mail_accounts	Konten ohne Secrets	UNIQUE(email_address, incoming_host)
mail_identities	Absenderidentitäten/Signaturen	account_id, is_default
provider_profiles	versionierte Providerdefaults	provider_key, version

Tabelle	Zweck	Wichtige Indizes
fetch_runs	Abrüfläufe und Berichtszähler	account_id, started_at, state
remote_message_identities	UIDL und Remote-Metadaten	UNIQUE(account_id, uidl)
messages	kanonische Nachrichtenidentität und Status	account_id, received_at, state
raw_messages	Dateipfad, Hash, Länge, Provenienz	UNIQUE(account_id, raw_sha256, raw_length)
parsed_messages	Parser-Version und normalisierte Inhalte	message_id, parser_version
message_addresses	From/To/Cc/Bcc/Reply-To	message_id, role, normalized_address
attachments	Metadaten und Hash	message_id, sha256, filename
outbox_items	SMTP-Warteschlange	state, next_attempt_at
sent_upload_jobs	IMAP-APPEND-Warteschlange	state, account_id
analysis_runs	versionierte Analysen	message_id, analyzer_id, analyzer_version
warning_findings	Einzelbefunde	analysis_run_id, severity, rule_id
brand_trust_profiles	lokale Markenprofile	profile_key, version
folders / message_folders	lokale Organisation	account_id, parent_id / message_id, folder_id
tags / message_tags	lokale Tags	name / message_id, tag_id
rules / rule_runs	declarative Regeln und Ausführung	enabled, priority / rule_id, message_id
message_search_fts	FTS5-Inhalte	virtueller FTS5-Index
schema_migrations	Upgradehistorie	version UNIQUE
audit_events	relevante Betriebsereignisse	correlation_id, occurred_at

6.1 Atomare Rohmail-Übernahme

8. Temporäre Datei im laufbezogenen Temp-Verzeichnis erstellen.
9. POP3-Stream bytengenau schreiben und gleichzeitig SHA-256 und Länge berechnen.
10. Datei flushen, schließen und bei unterstütztem Dateisystem Metadaten synchronisieren.
11. In einer SQLite-Transaktion Deduplizierungsindex prüfen und Importabsicht als TempStored registrieren.
12. Bei Duplikat: Transaktion mit DeduplicationEvent abschließen, Temp-Datei löschen.
13. Bei neuer Mail: Temp-Datei atomar in finalen Raw-Store-Pfad umbenennen; RawMessage und Message auf RawStored setzen; Transaktion committen.
14. Parser und Analyzer erst nach erfolgreicher kanonischer Übernahme starten.
15. Bei Start Inkonsistenzen zwischen Datei und DB über StartupReconciler in Quarantäne oder erneuten Verarbeitungszustand überführen.

6.2 Migrationen

- Jede Migration besitzt eine fortlaufende Nummer, Namen, Prüfsumme, Up-Skript und - sofern sicher möglich - Down-Skript.

- Migrationen werden vor Anwendung gegen eine Kopie der vorherigen unterstützten Version getestet.
- Vor Schemaänderungen wird ein konsistentes Backup verlangt oder automatisch erzeugt.
- Eine fehlgeschlagene Migration beendet den Start vor normalen Schreiboperationen und bietet Diagnose beziehungsweise Restore an.
- Der Sprung vom v0.3.0-Prototyp wird als explizite Basismigration dokumentiert; keine stillen Datenverluste oder Feldumdeutungen.

7. Konten, Providerprofile und Secret-Verwaltung

Baustein	Realisierung
AccountWizardViewModel	Schrittweise Eingabe von Adresse, Providerprofil, Servern, Ports, Sicherheitsmodi und SecretReference.
ProviderProfileService	Lädt mitgelieferte, signierte beziehungsweise gehashte JSON-Profil; kopiert Defaults in das editierbare Kontomodell.
ConnectionTestUseCase	DNS → TCP → TLS → Authentifizierung; getrennte Ergebnisse mit Dauer und Handlungshinweisen.
ISecretStore	Save/Get/Delete per SecretReference; Windows-Implementierung über Credential Manager oder DPAPI-geschützte Datei.
AccountRepository	Transaktionale Speicherung ohne Secrets; Deaktivierung verhindert neue Jobs, löscht aber keine Daten.
IdentityRepository	Mehrere Identitäten pro Konto, Defaultkennzeichen, Signatur als Text/HTML ohne aktive Inhalte.

Secret-Regel

Kein Passwort, Token oder App-Passwort darf in SQLite, appsettings.json, Logs, Crash Reports, Kontoexporten, Tests oder Beispielcode gespeichert werden. Nur Testprojekte dürfen synthetische Testsecrets verwenden; deren Werte werden durch automatisierte Logtests gesucht.

8. POP3-Abruf und „Get all“

Ablauf „Alle Mails abrufen“

Jede sichtbare Servermail wird geprüft; nur neue Originale werden kanonisch gespeichert.

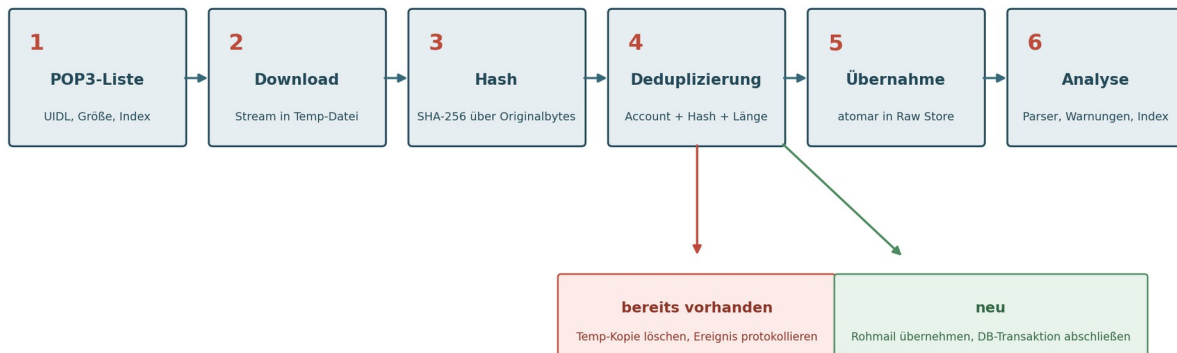


Abbildung 4: Vollständiger POP3-Abruf mit Hashvergleich

8.1 Normaler Abruf

16. Konto und Secret laden; Abruflauf Planned anlegen.
17. POP3 über SecureSocketOptions.SslOnConnect oder StartTlsRequired verbinden; Zertifikat regulär prüfen.
18. Authentifizieren und Capability/UIDL-Verfügbarkeit erfassen.
19. UIDLs und Größen in einem Batch laden; mit remote_message_identities abgleichen.
20. Nur unbekannte Einträge über denselben sicheren Stream-/Hash-/Temp-Mechanismus wie Get all verarbeiten.
21. Einzelfehler in FetchItem speichern und mit nächster Nachricht fortfahren, sofern Verbindung und Serverzustand dies erlauben.
22. Run-Zähler und Abschlussstatus persistieren; UI über IProgress<FetchProgressDto> aktualisieren.

8.2 Vollständiger Abruf „Get all“

23. Vor Beginn werden Serveranzahl, geschätzte Datenmenge, Speicherplatz und kontobezogene Löschoptionen angezeigt.
24. Jede sichtbare Nachricht wird vollständig geladen - unabhängig von bekannter UIDL.
25. UIDL dient nur der Provenienz; die Übernahmeentscheidung basiert auf AccountId, Rohhash und Länge.
26. Ein Duplikat erzeugt ein Ereignis und wird nicht erneut geparkt, sofern keine explizite Reanalyse angefordert wurde.
27. Der Lauf kann kontrolliert abgebrochen und später vollständig erneut gestartet werden.
28. Der Abschlussbericht enthält geprüft, neu, bekannt, fehlerhaft, Datenmenge, Laufzeit und verbleibende Warnungen.

8.3 POP3-Löschverhalten

- Default: Es wird niemals DeleteMessage/DeleteAllMessages verwendet.
- Eine spätere kontoabhängige Löschoption ist als SOLL-Anforderung vorbereitet, bleibt standardmäßig aus und verlangt eine deutliche Warnung sowie Sicherungsnachweis.
- Die Deduplizierung löscht ausschließlich die neu geladene temporäre lokale Kopie, niemals das kanonische Original und niemals die Servermail.

9. Hash-Deduplizierung und Bestandsprüfung

```
DedupKey = AccountId + SHA256(raw RFC-822 bytes) + RawLength
UNIQUE INDEX ux_raw_message_exact
ON raw_messages(account_id, raw_sha256, raw_length);
```

- Der Hash wird über exakt die Bytes gebildet, die als EML-Datei gespeichert würden. Eine erneute Serialisierung von MimeMessage ist für den Deduplizierungshash unzulässig.
- Message-ID, UIDL, Datum, Absender und semantische Fingerprints sind Diagnose- und Suchmerkmale, aber kein automatischer Löschbeweis.
- Der lokale Bestandsprüflauf gruppiert zunächst nur exakte Hash-/Längenübereinstimmungen. Vor Bereinigung wird ein Bericht erzeugt und eine kanonische Datei bestimmt.
- Die kanonische Auswahl bevorzugt einen gültigen, referenzierten Datensatz mit vollständiger Provenienz; alle abhängigen Ordner-, Tag- und Analysebezüge werden vor Entfernen einer Kopie zusammengeführt.
- Jeder Eingriff wird als DeduplicationEvent protokolliert und ist im Diagnosebericht nachvollziehbar.

10. MIME-Parsing, HTML-Sicherheit und Darstellung

Stufe	Verantwortung
Raw Loader	Liest Originaldatei read-only und prüft optional Hash/Länge.
MimeKit Parser	Dekodiert Header, Multipart, Transfer-Encoding, Text, HTML, Attachments und eingebettete Nachrichten.
Mapper	Überführt MimeKit-Typen in eigene ParsedMessage-/MimePart-DTOs.
Normalizer	Erzeugt bereinigten Plaintext, normalisierte Adressen und Linkbeobachtungen.
HTML Sanitizer	Entfernt aktive Inhalte, Formulare, Navigation, gefährliche Schemata und externe Ressourcen.
Render Host	Zeigt nur sanitiztes HTML in isoliertem WebView/Renderer; blockiert Navigation und Netzwerk.
Presentation	Bietet Text/HTML/Raw/Header/MIME-Ansichten, Zoom und klare Sicherheitszustände.

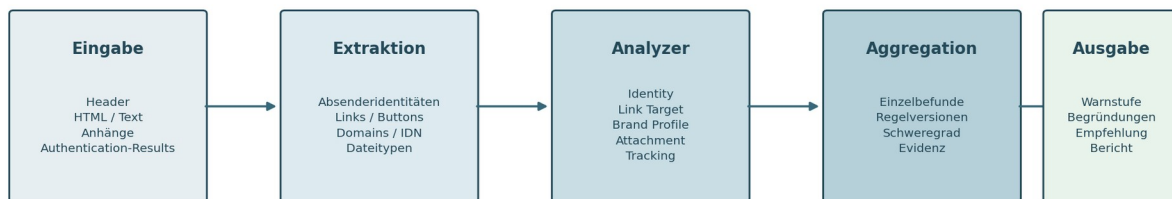
HTML-Sicherheitsgrenze

Der Renderer ist keine normale Browserinstanz. Er erhält keinen Zugriff auf Secrets, Datenbank, Dateisystem oder Hostobjekte. Jede Navigation und jeder Ressourcenabruf wird abgelehnt, solange nicht eine ausdrücklich freigegebene, separate Benutzeraktion erfolgt.

11. Lokales Warnsystem und Markenprofile

Lokales, erklärbares Warnsystem

Keine Blackbox: Jeder Befund besitzt Regel-ID, Evidenz, Version und verständliche Empfehlung.



Grundsatz: Das System warnt vor Auffälligkeiten, behauptet aber niemals garantierte Sicherheit.

Abbildung 5: Extraktion, Analyzer, Aggregation und erklärbare Ausgabe

11.1 Analyzer-Katalog für Version 1

Analyzer-ID	Prüfung	Ergebnis
sasd.security.identity.v1	From, Sender, Reply-To, Return-Path, Display Name und authentifizierte Domain	IdentityFinding
sasd.security.link-target.v1	sichtbarer Text/accessible name gegen tatsächliche Ziel-URL	LinkMismatchFinding
sasd.security.idn.v1	Unicode/Punycode, gemischte Skripte und confusable Zeichen	IdnFinding
sasd.security.transport-url.v1	IP-URL, HTTP, ungewöhnlicher Port, gefährliches Schema	UrlTransportFinding
sasd.security.brand-profile.v1	Zieldomain und Absender gegen lokale Markenprofile	BrandMismatchFinding

Analyzer-ID	Prüfung	Ergebnis
sasd.security.auth-results.v1	SPF/DKIM/DMARC aus Authentication-Results	AuthenticationFinding
sasd.security.tracking.v1	externe Ressourcen, 1x1-/unsichtbare Bilder, Redirectorlisten	TrackingFinding
sasd.security.attachment-type.v1	Dateiname, MIME, Extension und Magic Bytes	AttachmentTypeFinding

11.2 Domain- und Markenvergleich

- URLs werden syntaktisch geparkt, nicht per String-Suche. Hostname, Port, Schema, Pfad und Query werden getrennt gespeichert.
- Die registrierbare Domain wird über eine mit der Anwendung versionierte Public-Suffix-Liste bestimmt; fehlende oder private TLDs werden als unklar gekennzeichnet.
- BrandTrustProfile enthält primäre Domains, erlaubte Subdomains, bekannte Versand- und Trackingdienstleister, optionale Absenderdomains und Hinweise.
- Ein externer Dienstleister gilt nur dann als passend, wenn er explizit im aktiven, versionierten Profil steht. Die Mail selbst kann kein Vertrauen hinzufügen.
- Lokale Ausnahmen erzeugen eine neue Profilversion mit Begründung und Zeitstempel.

11.3 Warnstufen

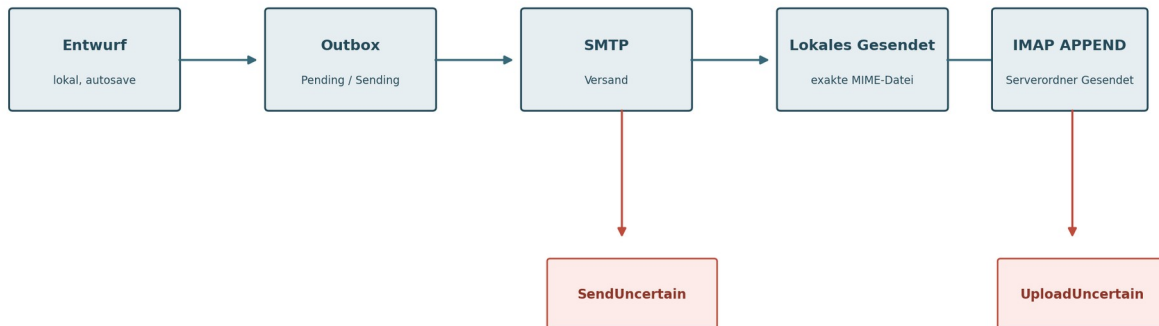
Stufe	Kriterien	UI-Verhalten
Information	technische Beobachtung ohne unmittelbare Auffälligkeit	neutral anzeigen; kein Block
Hinweis	ein schwacher oder erklärbarer Indikator	Symbol und Erläuterung; Linkvorschau
Erhöht	mehrere mittlere oder ein starker Indikator	Bestätigungsdialog vor Linköffnung; Gründe vollständig anzeigen
Hoch	kritische Kombination, z. B. Homograph + Identitätsabweichung + riskanter Anhang	Link/Öffnen standardmäßig blockiert; bewusste Mehrfachbestätigung oder Richtliniensperre

Keine Blackbox-Wahrscheinlichkeit

Version 1 verwendet keine behauptete Prozentwahrscheinlichkeit. Der Aggregator ordnet deterministische Regelbefunde einer Stufe zu und zeigt jeden Grund, die Evidenz, die Regelversion und eine konkrete Empfehlung.

12. Verfassen, Outbox, SMTP und IMAP-Gesendet-Upload

Versand und serverseitiger Gesendet-Upload



SMTP-Erfolg und IMAP-Erfolg bleiben getrennte Zustände. Unklare Ergebnisse werden niemals blind wiederholt.

Abbildung 6: Getrennte SMTP- und IMAP-Zustände

12.1 Verfassen und Entwürfe

- Der Editor arbeitet mit ComposeMessageViewModel und Draft DTO; Autosave erfolgt zeitgesteuert und zusätzlich bei Fokusverlust/Schließen.
- Text und HTML werden als multipart/alternative erzeugt; Anhänge und Inline-Bilder werden über verwaltete AttachmentDraft-Referenzen hinzugefügt.
- From/Identität, To, Cc, Bcc, Reply-To und Signatur werden vor Erzeugung der finalen MIME-Nachricht validiert.
- Die Warnungen „leerer Betreff“ und „Anhang erwähnt, aber keiner vorhanden“ sind nicht blockierend, verlangen aber bewusste Bestätigung.

12.2 SMTP und SendUncertain

- Die finale MIME-Datei wird vor dem ersten Sendeversuch unveränderlich in der Outbox gespeichert.
- Temporäre SMTP-Fehler können nach begrenztem exponentiellem Backoff wiederholt werden; permanente 5xx-Fehler werden nicht automatisch wiederholt.
- Bricht die Verbindung in einer Phase ab, in der die Serverannahme unklar ist, wird SendUncertain gesetzt. Ein automatischer Blind-Retry ist verboten.
- Die UI zeigt SMTP-Status, letzte Serverantwort, Zeitpunkt und sichere Handlungsoption getrennt vom IMAP-Upload.

12.3 IMAP APPEND

- Nach sicherem SMTP-Erfolg wird dieselbe MIME-Datei als lokale Gesendet-Mail registriert.
- Der SentFolderResolver ermittelt \Sent Special-Use, Providerprofil oder Nutzerauswahl.
- AppendAsync lädt die vollständige Nachricht mit Seen-Flag hoch; APPENDUID wird gespeichert, sofern der Server UIDPLUS liefert.

- Bei UploadUncertain wird vor Retry im Zielordner nach stabilen Merkmalen gesucht. SMTP wird niemals erneut ausgeführt, nur weil APPEND scheiterte.
- Die Warteschlange zeigt Pending, Uploading, Uploaded, UploadUncertain, RetryRequired und FailedPermanent.

13. Organisation, Suche, Regeln und Anhänge

13.1 Lokale Ordner und Tags

- Systemordner werden als unveränderliche FolderKind-Werte geführt; Anzeigenamen sind lokalisierbar.
- Lokale Ordner bilden eine Baumstruktur pro Konto oder globalem Workspace. Nachrichten können mehreren virtuellen Ansichten, aber genau einer primären lokalen Ablage zugeordnet sein.
- Tags sind kontoübergreifend möglich und besitzen Name, optionale Farbe, Symbol und Sortierung; Farbe ist nie alleiniger Informationsträger.

13.2 Volltextsuche

- FTS5 indexiert Betreff, normalisierte Absender/Empfänger, bereinigten Plaintext und Attachment-Dateinamen.
- Der Suchindex ist rekonstruierbar und enthält keine alleinige Quelle fachlicher Wahrheit.
- Suche unterstützt strukturierte Filter für Konto, Ordner, Datum, gelesen, markiert, Tag und Anhang.
- UI-Abfragen sind paginiert/virtualisiert; Bodies und Anhänge werden nicht für jede Listenzeile geladen.
- Indexierungsfehler markieren die Nachricht als SearchIndexPending und blockieren nicht das Lesen.

13.3 Regelengine

- Regeln bestehen aus versionierten Bedingungen und whitelist-basierten Aktionen.
- Bedingungen: Absender, Empfänger, Betreff, Header, Text enthält, Tag, Attachment vorhanden.
- Aktionen: lokaler Ordner, Tag, gelesen/ungelesen, markieren, RequiresReview.
- Keine externe Skript-, Shell- oder dynamische CLR-Ausführung in Version 1.
- Jeder Regellauf speichert Regelversion, Ergebnis und ausgeführte Aktionen.

13.4 Anhänge

- Anhänge werden nicht automatisch ausgeführt oder in temporären Standardordnern geöffnet.
- Speichern verwendet sichere Dateinamensnormalisierung und behandelt Kollisionen über Nachfragen oder deterministische Suffixe.
- Magic-Byte-Erkennung ergänzt, aber ersetzt nicht den deklarierten MIME-Type; Abweichungen werden gewarnt.
- SHA-256 ist obligatorisch; SHA-512 kann bei Bedarf nachberechnet werden.
- Inline-CID-Ressourcen werden separat klassifiziert und nur im isolierten Renderer bereitgestellt.

14. Scientific Foundation und Message Laboratory

14.1 Message Laboratory

Register	Inhalt
Nachricht	lesbare Text-/HTML-Ansicht, Adressen, Betreff, Datum, Anhänge
Analyse	Warnstufe, Analyserresultate, Link-/Domainbefunde, Attachmentbefunde
Rohdaten	vollständige Header, Raw-Eml-Ansicht, dekodierter Body, MIME-Baum
Historie	Abrufe, Provenienz, Parsing, Analyseläufe, Reprocessing, Exporte und manuelle Ausnahmen

14.2 Reproduzierbare Analysis Runs

- AnalyzerId, AnalyzerVersion und ResultSchemaVersion sind stabil und unabhängig vom CLR-Typnamen.
- InputHash entspricht dem Rohmailhash oder dem explizit dokumentierten abgeleiteten Eingabehash.
- ConfigurationHash umfasst relevante Regel-, Markenprofil- und Analyserparameter.
- Ergebnisse werden als strukturiertes JSON plus normalisierte Summary-/Finding-Tabellen gespeichert.
- Neue Analyzer-Versionen überschreiben alte Resultate nicht; Vergleich und Superseded-Markierung bleiben möglich.

14.3 Datensätze und Exporte

- Dataset Builder ist als SOLL-Funktion implementierbar, sobald Kernabruf und Warnsystem stabil sind.
- Ein Dataset speichert Kriterien und explizite Mitglieder, damit dynamische Mailbestände das Ergebnis nicht still verändern.
- Exportmanifest enthält Softwareversion, Schema, Message-Hashes, Analysis Runs, Pseudonymisierungsprofil und Erstellzeit.
- Mögliche Formate: CSV, JSON, JSONL, SQLite und Markdown. EML-Originale werden nur nach ausdrücklicher Auswahl beigefügt.

15. Extension Model und Plugin-Vorbereitung

Vorbereitung des Erweiterungsmodells



Version 1 registriert Module per Dependency Injection; ein offenes DLL-Plugin-System wird bewusst noch nicht ausgeliefert.

Abbildung 7: Stabile Verträge ohne offenes DLL-Laden in Version 1

```
public interface IMessageAnalyzer
{
    ExtensionDescriptor Descriptor { get; }

    Task<AnalysisResultEnvelope> AnalyzeAsync(
        AnalysisInput input,
        AnalyzerContext context,
        CancellationToken cancellationToken);
}

public sealed record AnalysisResultEnvelope(
    string ExtensionId,
    string ExtensionVersion,
    string ResultSchemaVersion,
    AnalysisStatus Status,
    JsonDocument Result,
    IReadOnlyList<WarningFindingDto> Findings);
```

- ExtensionModel bleibt klein, binär stabil und frei von WinForms, MailKit und SQLite.
- Eingebaute Analyzer und Processor verwenden dieselben Verträge wie spätere Plugins.
- Host registriert Module in definierter Reihenfolge; zyklische Abhängigkeiten zwischen Modulen sind verboten.
- Persistenz speichert IDs, Versionen und Schema-Versionen, nicht konkrete Klassen oder Assemblynamen.
- Fremde DLLs, UI-Plugins, Marketplace, Auto-Update und Sandbox sind ausdrücklich nicht Bestandteil von Version 1.

- Für späteren nicht vertrauenswürdigen Code wird ein separater Prozess mit IPC statt In-Process-DLL als bevorzugte Option dokumentiert.

16. Presentation Core und WinForms-Oberfläche

16.1 Grundlayout

- Kopfbereich: Profilbild, aktives Konto, Online-/Abrufstatus und zentrale Aktionen.
- Linke Navigation: Konten, Systemordner, lokale Ordner, Tags und Workspaces.
- Mittlere Nachrichtenliste: virtualisiert, sortier- und filterbar, Status nicht nur per Farbe.
- Rechter Lesebereich: Nachricht/Analyse/Rohdaten/Historie, responsive über SplitContainer und gespeicherte Breiten.
- Unterer Statusbereich: laufender Job, Fortschritt, Fehleranzahl, Abbrechen und Detailbericht.

16.2 Presentation Core

ViewModel/Port	Verantwortung
MainShellViewModel	Navigation, aktiver Workspace, globaler Status
AccountTreeViewModel	Konten/Ordner/Tags ohne WinForms-Typen
MessageListViewModel	Paging, Sortierung, Filter, Auswahl
MessageDetailsViewModel	Text/HTML/Raw/Analyse/Historie
GetAllMessagesViewModel	Start, Fortschritt, Abbruch, Bericht
ComposeMessageViewModel	Entwurf, Validierung, Anhänge, Versand
SentUploadQueueViewModel	Uploadstatus und Retry
IUserService	Info, Fehler, Bestätigung
IFileDialogService	Öffnen/Speichern ohne UI-Kopplung
ILinkOpenService	Warnbewertung und externes Öffnen
INavigationService	Workspace- und Detailnavigation

16.3 WinForms-Partial-Class-Regeln

MainForm.cs	Konstruktor, Lebenszyklus, DI
MainForm.Designer.cs	ausschließlich Visual-Studio-Designer
MainForm.resx	Ressourcen
MainForm.Events.cs	kleine Eventhandler, die ViewModel-Commands aufrufen
MainForm.Binding.cs	UI-Binding und Zustandsprojektion
MainForm.Presentation.cs	rein visuelle Hilfsmethoden

- Kein POP3-, SMTP-, IMAP-, SQL-, Hash-, Parser- oder Analyzer-Code in Forms.

- Eventhandler sind async void nur an der UI-Ereignisgrenze; Fehler werden zentral über SafeUiCommand/ExceptionHandler behandelt.
- Designerfelder werden nie doppelt deklariert; Designerdateien werden nicht manuell formatiert oder mit Geschäftslogik erweitert.
- Große Bereiche werden als UserControls mit eigenem ViewModel geteilt.
- Layout, Theme und Dichte werden über Konfigurationsmodelle getrennt gehalten, damit frühere Designstudien später als Templates umgesetzt werden können.

17. Logging, Diagnose, Backup und Wiederherstellung

17.1 Strukturierte Logs

Property	Bedeutung
CorrelationId	übergreifender Use-Case-/UI-Vorgang
AccountId	betroffenes Konto, niemals Adresse als Pflichtfeld
MessageId	interne Nachricht
FetchRunId	Abrufauflauf
OutboxId / UploadId	Versand- beziehungsweise Uploadjob
AnalysisRunId	Analyselauf
EventId	stabile Ereignisnummer
ErrorCode	stabiler fachlicher/technischer Fehlercode

- Bodies, vollständige Betreffzeilen, Adressen, Secrets und Token werden nicht standardmäßig geloggt.
- Sensible Werte werden bereits vor Übergabe an den Logger maskiert; ein automatisierter Test sucht bekannte Testmarker.
- Protokolldateien rotieren nach Größe/Alter und können über die Anwendung bereinigt werden.

17.2 Diagnosepaket

- Vorschau aller einzuschließenden Dateien und Kategorien vor Export.
- Standard: Versionsinformationen, anonymisierte Konfiguration, Migrationen, Integritätsresultate, ausgewählte Logs, keine Bodies/EMLs/Secrets.
- Optionale Pseudonymisierung stabiler IDs für Konto, Adresse, Host und IP.
- Manifest mit Hashes, Erstellzeit und angewendetem Redaktionsprofil.

17.3 Backup und Restore

29. Neue Schreibjobs anhalten beziehungsweise in sicheren Zustand bringen.
30. SQLite-Onlinebackup oder kontrollierter Snapshot mit WAL-Konsolidierung erzeugen.
31. Referenzierte Rohmaildateien, Provider-/Markenprofile und nicht geheime Einstellungen kopieren.
32. Manifest mit Version, Dateilänge und SHA-256 erzeugen; stichprobenlose vollständige Validierung durchführen.

33. Restore nur in leeres Profil oder nach expliziter Sicherung/Bestätigung; Secrets müssen separat erneut zugeordnet werden.
34. Nach Restore Datenbankintegrität, Fremdschlüssel, Rohdateien und Suchindex prüfen; FTS5 kann neu aufgebaut werden.

18. Fehlerbehandlung, Wiederanlauf und Parallelität

Fehlerklasse	Codebeispiele	Behandlung
Konfiguration	CFG-VALIDATION, CFG-SECRET-MISSING	Speichern blockieren; Feldbezug und Korrekturhinweis anzeigen.
DNS/TCP/TLS	NET-DNS, NET-CONNECT, TLS-CERTIFICATE	Schrittgenaue Diagnose; keine stille Sicherheitsabsenkung.
Authentifizierung	AUTH-REJECTED, AUTH-MECHANISM	Konto nicht automatisch sperren; Providerhinweis und Secretprüfung anbieten.
POP3 Nachricht	POP3-ITEM-FAILED	FetchItem als fehlerhaft, Lauf fortsetzen, Retry einzeln ermöglichen.
Persistenz	DB-CONSTRAINT, RAW-MISSING, STORAGE-FULL	Transaktion rollback; Job stoppen, Datenkonsistenz priorisieren.
Parsing	MIME-PARSE-WARNING, MIME-PARSE-FAILED	Rohmail bleibt zugänglich; Teilresultate/Warnungen speichern; Reprocessing anbieten.
SMTP	SMTP-TEMPORARY, SMTP-PERMANENT, SMTP-UNCERTAIN	temporär mit Backoff; permanent ohne Auto-Retry; unklar nie blind wiederholen.
IMAP APPEND	IMAP-FOLDER, IMAP-APPEND, IMAP-UNCERTAIN	separate Uploadqueue; Zielordner prüfen; vor Retry deduplizieren.
Analyse	ANALYZER-FAILED, RESULT-SCHEMA	anderen Analyzer nicht blockieren; Lauf mit Fehler und Diagnose speichern.
UI	UI-CANCELLED, UI-UNHANDLED	Cancellation normal behandeln; unerwartete Ausnahme zentral protokollieren und sichere Benutzeranzeige.

18.1 Wiederanlauf

- Beim Start werden Jobs in transienten Zuständen geprüft: Fetching, TempStored, Sending, Uploading und Running.
- Fetching/TempStored wird anhand Temp-Datei, Rohdatei und DB-Eintrag rekonstruiert oder sicher abgebrochen.
- Sending wird nicht automatisch auf Pending zurückgesetzt; bei möglicher Serverannahme wird SendUncertain verwendet.
- Uploading wird UploadUncertain, wenn nicht eindeutig nachweisbar ist, dass APPEND fehlgeschlagen ist.

- Running Analysis wird Failed/Interrupted; ein neuer Lauf kann explizit gestartet werden.
- Alle Wiederanlaufentscheidungen erzeugen AuditEvent und verständliche UI-Mitteilung.

18.2 Parallelität

- Pro Konto läuft höchstens eine POP3-Sitzung gleichzeitig. Unterschiedliche Konten dürfen parallel verarbeitet werden, sofern Ressourcenlimits eingehalten werden.
- Deduplizierungs-UNIQUE-Index und Transaktionen schützen auch vor konkurrierenden Importen.
- Parsing und Analyse können über eine begrenzte Worker-Queue parallelisiert werden; Anzahl ist konfigurierbar und standardmäßig konservativ.
- Datenbankzugriffe verwenden kurze Transaktionen. Langsame Netzwerk- oder Dateivorgänge dürfen keine SQLite-Schreibtransaktion offen halten.
- UI erhält immutable Snapshots/DTOs; Hintergrundthreads greifen nicht direkt auf Controls zu.

19. Nichtfunktionale Umsetzung und Qualitätsziele

Qualitätsbereich	Technische Festlegung	Mess-/Nachweisverfahren
Zuverlässigkeit	atomare Rohübernahme, explizite Zustände, Idempotenz, Startup-Reconciliation	Crash-/Abbruchtests an definierten Übergängen
Performance	100.000 Nachrichten, Paging, FTS5, streambasierter Hash, Lazy Loading	synthetischer 100k-Datensatz; Antwortzeiten messen
Sicherheit	TLS erforderlich, Secrets außerhalb DB, HTML untrusted, keine Onlineanalyse default	Security-Tests, Log-Secret-Scan, Renderer-Fuzz-Fixtures
Datenschutz	lokale Verarbeitung, Exportvorschau, Pseudonymisierung, keine Pflichttelemetrie	Konfigurationsprüfung und Diagnosepaket-Review
Bedienbarkeit	Tastatur, 150 % DPI, sichtbarer Fokus, erklärbare Fehler	manuelle Accessibility-Checkliste + UI-Smoke-Tests
Wartbarkeit	XML-Dokumentation, ADRs, kleine Projekte, zentrale Paketversionen	Compiler-Dokumentationswarnungen, Architekturtests, Review-Checkliste
Testbarkeit	Ports/Adapter, Fake-Mailserver, isolierte Profile	automatisierte Unit-/Integration-/Architekturtests
Portierbarkeit	kein WinForms außerhalb Presentation/Host	Referenz- und Namespace-Architekturtests
Interoperabilität	EML/MIME/POP3/SMTP/IMAP APPEND/SQLite	Roundtrip- und kontrollierte Protokolltests
Dokumentation	Installations-, Entwickler-, Test-, Admin- und Restore-Dokumente	Release-Gate: Dokumente aktuell und gegen Version geprüft

19.1 Performance-Budgets

Vorgang	Zielwert auf Referenzsystem	Hinweis
Start bis Hauptfenster	< 5 Sekunden ohne Migration/Recovery	Hintergrundindizes verzögert laden.
Nachrichtenliste erste Seite	< 1 Sekunde bei 100.000 Nachrichten	nur Summaryfelder, Paging.
Lokale Standardsuche	< 2 Sekunden für typische Query	FTS5 warm; komplexe Filter dürfen progressiv laden.
Öffnen einer normalen Mail	< 1 Sekunde ohne großen Attachment-Download	Rohdatei lokal; Analyse ggf. nachladen.
UI-Reaktion	keine Blockade > 200 ms auf UI-Thread	I/O stets async; CPU-Arbeit im Worker.
Speicher	kein vollständiges Laden großer Mailbox oder aller Bodies	streaming, paging, bounded caches.

Referenzsystem

Die endgültigen Hardwaredaten des Abnahmesystems werden vor Performance-Abnahme festgeschrieben. Zielwerte sind keine Garantie für jeden Rechner, sondern ein reproduzierbares Qualitätsbudget.

20. Teststrategie und Abnahme

20.1 Testpyramide

Ebene	Inhalt	Ausführung
Unit	Domainregeln, Validatoren, Aggregatoren, URL-/IDN-Analyse, ViewModels	bei jedem Build
Component	Parser, Analyzer, Repository gegen temporäre SQLite-DB, Raw Store	bei jedem Pull Request
Integration	Application Use Cases mit realen Adaptern/Fakes, kontrollierte Protokollserver	CI und vor Release
Architecture	verbotene Referenzen, Namespaces, Package-Abhängigkeiten	bei jedem Build
Security	HTML-/URL-Fixtures, Secret-Scan, Pfad-/Dateinametests	CI und Release
Performance	100k-Datensatz, Streaming großer EMLs, Suchindex	Nightly/Release Candidate
UI Smoke	Start, Navigation, Tastatur, 100/150 % DPI	Release Candidate
Acceptance	AT-001 bis AT-017	formale Release-Abnahme

20.2 Testserver und Testdaten

- Keine automatisierten Tests verwenden produktive GMX-/IONOS-Postfächer.
- POP3/SMTP/IMAP werden über kontrollierte lokale Testserver, Container oder deterministische Fake-Adapter geprüft.
- Die EML-Fixture-Sammlung enthält Plaintext, HTML, Multipart, defekte Encodings, Inline-CID, große Attachments, Linkmismatch, IDN/Punycode, Authentication-Results und riskante Dateinamen.
- Jede Sicherheitsregel erhält mindestens einen positiven, einen negativen und einen legitimen Ausnahmefall.
- Testmails enthalten ausschließlich synthetische, nicht personenbezogene Inhalte.

20.3 Abnahmeszenarien

ID	Szenario	Testumgebung	Nachweis
AT-001	Zwei Konten (GMX/IONOS oder Testäquivalente) konfigurieren und getrennt testen.	kontrollierter POP3-Testserver + isoliertes Profil	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-002	Normaler POP3-Abruf übernimmt neue Mails und überspringt bekannte UIDs.	kontrollierter POP3-Testserver + isoliertes Profil	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-003	„Get all“ lädt den gesamten Serverbestand, ergänzt fehlende Mails und erzeugt keine exakten lokalen Duplikate.	kontrollierter POP3-Testserver + isoliertes Profil	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-004	Wiederholter „Get all“-Lauf bei unverändertem Serverbestand erhöht die Zahl kanonischer Mails nicht.	kontrollierter POP3-Testserver + isoliertes Profil	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-005	Servermails bleiben nach Standardabruf erhalten.	kontrollierter POP3-Testserver + isoliertes Profil	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-006	Eine HTML-Testmail mit sichtbarem Sparkassenlink und fremdem href erzeugt eine erklärbare Warnung.	versionierte EML-Fixtures + Analysis/Persistence	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-007	Punycode-/Homograph-, IP-URL-, HTTP- und Reply-To-Abweichungen werden einzeln angezeigt.	versionierte EML-Fixtures + Analysis/Persistence	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-008	Eine legitime, im Markenprofil gepflegte Dienstleisterdomain lässt sich als nachvollziehbare Ausnahme modellieren.	versionierte EML-Fixtures + Analysis/Persistence	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-009	SMTP-Versand erstellt lokale Gesendet-Mail; IMAP APPEND macht sie für einen zweiten IMAP-Client sichtbar.	kontrollierter SMTP-/IMAP-Testserver	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.

ID	Szenario	Testumgebung	Nachweis
AT-010	Ein Verbindungsabbruch nach möglichem APPEND führt zu UploadUncertain und nicht zu blindem Duplikatupload.	kontrollierter SMTP-/IMAP-Testserver	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-011	Plaintext, HTML, Multipart, Inline-Ressourcen und Anhänge werden sicher dargestellt.	versionierte EML-Fixtures + Analysis/Persistence	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-012	Remote-Ressourcen und Skripte werden standardmäßig nicht ausgeführt.	versionierte EML-Fixtures + Analysis/Persistence	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-013	Rohmail, Hash, Provenienz und Analysis Run bleiben nach Neustart erhalten.	versionierte EML-Fixtures + Analysis/Persistence	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-014	Backup lässt sich in ein leeres Profil wiederherstellen und besteht Integritätsprüfung.	isoliertes Quell- und Zielprofil	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-015	Architekturtests bestätigen, dass WinForms weder MailKit noch SQLite direkt referenziert.	Architekturtestprojekt	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-016	Kernabläufe funktionieren bei 150 % Windows-Skalierung und per Tastatur.	Windows UI bei 100 % und 150 % DPI	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.
AT-017	Automatisierte Tests werden nach Testhandbuch in Visual Studio und per Kommandozeile ausgeführt.	Visual Studio und dotnet CLI	Automatisiert soweit möglich; Ergebnisprotokoll Bestandteil der Releaseakte.

21. Build, Deployment und Betrieb

21.1 Entwicklungsumgebung

- Für 0.3.1 und 0.4.0: Visual Studio 2022 mit aktuellem .NET-8-SDK und Workload „.NET-Desktopentwicklung“. Für die Migration auf .NET 10 ist eine dafür unterstützte Visual-Studio-Version einzusetzen.
- .NET SDK 10.0.x; die SDK-Version wird über global.json gepinnt.
- Central Package Management über Directory.Packages.props; Restore-Lockfiles oder kontrollierte Renovate/Dependabot-Prüfung.
- Nullable, implicit usings, analyzers, warnings-as-errors für Produktprojekte und XML-Dokumentationsdateien aktiviert.
- Buildkonfigurationen Debug und Release; keine Secrets in User Secrets für produktive Konten, sondern Secret-Store-Adapter.

21.2 Build-Pipeline

```
dotnet restore --locked-mode

dotnet build Sasd.MailWorkbench.sln -c Release --no-restore

dotnet test Sasd.MailWorkbench.sln -c Release --no-build --collect:"XPlat Code Coverage"

dotnet publish src/Sasd.MailWorkbench.Host.WinForms -c Release -r win-x64 --self-contained false
```

- CI prüft Formatierung, Build, Tests, Architekturregeln, Paket-Sicherheitswarnungen und Dokumentationsartefakte.
- Releasepakete werden reproduzierbar versioniert; AssemblyVersion bleibt kompatibel, File/InformationalVersion enthält SemVer und Commit-ID.
- Installer-Technologie wird im Implementierungsmeilenstein festgelegt; bevorzugt MSIX oder signierter MSI/Bootstrapper. Portable ZIP ist nicht primäres Sicherheitsmodell.
- Code Signing und Installer Signing sind für eine externe Veröffentlichung vorgesehen; lokale Entwicklungsbuids bleiben eindeutig gekennzeichnet.

21.3 Konfiguration

Datei/Quelle	Inhalt	Secret?
appsettings.json	Logging, Featureflags, Standardpfade, Limits	Nein
provider-profiles/*.json	GMX/IONOS und weitere Defaults	Nein
brand-profiles/*.json	lokale Marken-/Vertrauensprofile	Nein
ui-settings.json	Theme, Layout, Dichte, Fensterzustand	Nein
SQLite	fachliche Daten, Zustände, Indizes	Nein, keine Passwörter
Windows Secret Store	Passwörter/App-Passwörter/Tokens	Ja

22. Umsetzungsplan bis Version 1.0.0

Meilenstein	Inhalt	Exit-Kriterien
M0 - Konsolidierung	Pflichtenheft freigeben, v0.3.0 inventarisieren, Namespace-/Repo-Entscheidung, .NET-10-Migration	Build grün; alte Tests grün; ADR-009 freigegeben.
M1 - Architekturgrundlage	Solutionprojekte, DI, Architekturtests, Error/Result Contracts, Logging	verbotene Referenzen automatisiert gesperrt.
M2 - Persistenz 1.0	Schema, Migrationen, Raw Store, atomare Übernahme, FTS5-Grundlage	Crash-/Recovery-Tests grün.
M3 - Konten und POP3	Providerprofile, Secrets, Connection Test, normaler Abruf	AT-001/002/005 grün.

Meilenstein	Inhalt	Exit-Kriterien
M4 - Get all und Deduplizierung	vollständiger Abruf, Hash, Temp, Bericht, Bestandsprüfung	AT-003/004 grün.
M5 - MIME und Darstellung	MimeKit-Adapter, Parser, sichere HTML-Policy, Raw/MIME-Ansicht	AT-011/012 grün.
M6 - Warnsystem	Link/Identity/IDN/Brand/Auth/Attachment Analyzer	AT-006/007/008 grün.
M7 - Versand	Compose, Draft, Outbox, SMTP, lokales Gesendet, IMAP APPEND	AT-009/010 grün.
M8 - Organisation	Ordner, Tags, Suche, Regeln, Attachment Save	100k-/Suchtests grün.
M9 - Scientific/Extension	Message Laboratory, Analysis Runs, Extension Contracts, Reportgrundlage	AT-013/015 grün.
M10 - Betrieb/Release	Backup/Restore, Diagnose, Accessibility, Handbücher, Installer	AT-014/016/017 und alle MUSS grün.

23. Rückverfolgbarkeit aller Anforderungen

Dieses Kapitel behandelt jede funktionale und nichtfunktionale Anforderung des Lastenhefts einzeln. Der Wortlaut wird übernommen; Realisierung, betroffene Daten, Fehlerfälle und Verifikation werden verbindlich zugeordnet.

23.1 Konten und Identitäten

LF-KTO-001 [MUSS] - Die Anwendung muss mehrere POP3-/SMTP-Konten eines lokalen Benutzers verwalten können.

Technische Realisierung	Konten werden durch eine unveränderliche AccountId getrennt. Sämtliche Nachrichten-, Abruf- und Versanddatensätze tragen diese AccountId als Fremdschlüssel; Konten können parallel konfiguriert, aber pro Konto serialisiert abgerufen werden. Primär betroffen sind Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Die fachlichen Daten werden über MailAccount, MailIdentity, ProviderProfile, SecretReference abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MailAccount, MailIdentity, ProviderProfile, SecretReference. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	falscher Port/Sicherheitsmodus; Providerprofil veraltet; Konto deaktiviert bei laufendem Job.
Verifikation	Unit-Test der Validierung und ViewModel-Logik. Fachlicher Nachweis: Mindestens zwei Konten lassen sich getrennt konfigurieren, aktivieren und abrufen.

LF-KTO-002 [MUSS] - Für jedes Konto müssen Anzeigename, vollständige E-Mail-Adresse, Benutzername, Eingangsserver, Ausgangsserver, Ports und Sicherheitsmodi konfigurierbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Die fachlichen Daten werden über MailAccount, MailIdentity, ProviderProfile, SecretReference abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MailAccount, MailIdentity, ProviderProfile, SecretReference. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	falscher Port/Sicherheitsmodus; Providerprofil veraltet; Konto deaktiviert bei laufendem Job.
Verifikation	Unit-Test der Validierung und ViewModel-Logik. Fachlicher Nachweis: Alle Felder sind editierbar, validiert und werden ohne Geheimnisse exportiert.

LF-KTO-003 [MUSS] - Die Anwendung muss Providerprofile für GMX und IONOS anbieten, ohne manuelle Konfiguration zu verhindern.

Technische Realisierung	Providerprofile werden als versionierte JSON-Ressourcen ausgeliefert und beim Kontoassistenten nur als Vorschlag angewendet. Nach dem Kopieren in das Kontomodell bleiben Host, Port und Security-Modus vollständig editierbar. Primär betroffen sind Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Die fachlichen Daten werden über MailAccount, MailIdentity, ProviderProfile, SecretReference abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MailAccount, MailIdentity, ProviderProfile, SecretReference. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	falscher Port/Sicherheitsmodus; Providerprofil veraltet; Konto deaktiviert bei laufendem Job.
Verifikation	Unit-Test der Validierung und ViewModel-Logik. Fachlicher Nachweis: GMX und IONOS sind auswählbar; alle Werte bleiben manuell überschreibbar.

LF-KTO-004 [MUSS] - Vor dem Speichern eines Kontos muss ein nachvollziehbarer Verbindungstest möglich sein.

Technische Realisierung	Der ConnectionTestUseCase führt DNS-Auflösung, TCP-Verbindung, TLS-Handshake und Authentifizierung als getrennte Schritte aus. Jeder Schritt erzeugt Dauer, Status, technische Ursache und benutzerverständliche Empfehlung. Primär betroffen sind Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Die fachlichen Daten werden über MailAccount, MailIdentity, ProviderProfile, SecretReference abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MailAccount, MailIdentity, ProviderProfile, SecretReference. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	falscher Port/Sicherheitsmodus; Providerprofil veraltet; Konto deaktiviert bei laufendem Job.
Verifikation	Unit-Test der Validierung und ViewModel-Logik. Fachlicher Nachweis: DNS, TCP, TLS und Authentifizierung werden getrennt als erfolgreich oder fehlerhaft angezeigt.

LF-KTO-005 [MUSS] - Ein Konto muss deaktiviert werden können, ohne lokale Nachrichten zu verlieren.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Die fachlichen Daten werden über MailAccount, MailIdentity, ProviderProfile, SecretReference abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MailAccount, MailIdentity, ProviderProfile, SecretReference. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	falscher Port/Sicherheitsmodus; Providerprofil veraltet; Konto deaktiviert bei laufendem Job.
Verifikation	Unit-Test der Validierung und ViewModel-Logik. Fachlicher Nachweis: Deaktiviertes Konto wird nicht automatisch abgerufen; Daten bleiben sichtbar.

LF-KTO-006 [SOLL] - Pro Konto sollen mehrere Absenderidentitäten und Signaturen vorbereitet werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Die fachlichen Daten werden über MailAccount, MailIdentity, ProviderProfile, SecretReference abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MailAccount, MailIdentity, ProviderProfile, SecretReference. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	falscher Port/Sicherheitsmodus; Providerprofil veraltet; Konto deaktiviert bei laufendem Job.
Verifikation	Unit-Test der Validierung und ViewModel-Logik. Fachlicher Nachweis: Mindestens eine zusätzliche Identität kann gespeichert und beim Verfassen gewählt werden.

LF-KTO-007 [MUSS] - Geheimnisse dürfen nicht Bestandteil normaler Kontoexporte sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Die fachlichen Daten werden über MailAccount, MailIdentity, ProviderProfile, SecretReference abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MailAccount, MailIdentity, ProviderProfile, SecretReference. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application / Security / Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	falscher Port/Sicherheitsmodus; Providerprofil veraltet; Konto deaktiviert bei laufendem Job.
Verifikation	Unit-Test der Validierung und ViewModel-Logik. Fachlicher Nachweis: Exportdatei enthält weder Passwort noch Token oder entschlüsselbare Ersatzwerte.

23.2 POP3-Abruf

LF-ABR-001 [MUSS] - Der normale Abruf muss über POP3S beziehungsweise POP3 mit zwingendem TLS erfolgen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Unverschlüsselte Verbindungen werden nicht automatisch verwendet.

LF-ABR-002 [MUSS] - Der Befehl „Neue Mails abrufen“ muss UIDL und den lokalen Index verwenden, um nur noch nicht bekannte Nachrichten abzurufen.

Technische Realisierung	Der schnelle Abruf liest die UIDL-Liste einmal pro Sitzung, gleicht sie über einen eindeutigen Index (AccountId, Uidl) ab und lädt nur unbekannte Einträge vollständig. UIDL wird niemals als globale, kontoübergreifende Identität behandelt. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Bei unverändertem Serverbestand werden keine vollständigen bekannten Mails erneut gespeichert.

LF-ABR-003 [MUSS] - Der Befehl „Alle Mails abrufen“ muss alle auf dem POP3-Server sichtbaren Nachrichten vollständig herunterladen und lokal vergleichen.

Technische Realisierung	DownloadAllMessagesUseCase lädt unabhängig vom UIDL-Bekanntheitsstatus jede sichtbare Nachricht in einen temporären Stream, berechnet während des Schreibens den Rohhash und entscheidet erst danach über kanonische Übernahme oder Verwerfen der Temp-Datei. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Auch bekannte UIDLs werden vollständig geprüft; Bericht weist geprüft, neu, bekannt und fehlerhaft aus.

LF-ABR-004 [MUSS] - Der vollständige Abruf muss abbrechbar sein.

Technische Realisierung	DownloadAllMessagesUseCase lädt unabhängig vom UIDL-Bekanntheitsstatus jede sichtbare Nachricht in einen temporären Stream, berechnet während des Schreibens den Rohhash und entscheidet erst danach über kanonische Übernahme oder Verwerfen der Temp-Datei. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.

Technische Realisierung	DownloadAllMessagesUseCase lädt unabhängig vom UIDL-Bekanntheitsstatus jede sichtbare Nachricht in einen temporären Stream, berechnet während des Schreibens den Rohhash und entscheidet erst danach über kanonische Übernahme oder Verwerfen der Temp-Datei. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Abbruch beendet den Lauf kontrolliert; bereits vollständig übernommene Mails bleiben konsistent.

LF-ABR-005 [MUSS] - Ein abgebrochener oder unterbrochener Abruf muss ohne Datenverlust erneut gestartet werden können.

Technische Realisierung	Alle langen Operationen akzeptieren CancellationToken. Abbruch wird nur zwischen atomaren Schritten wirksam; bereits finalisierte Nachrichten bleiben gültig, der aktuelle temporäre Datensatz wird als Cancelled markiert und beim nächsten Start bereinigt. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Der Folgelauf erzeugt keine doppelten lokalen Nachrichten.

LF-ABR-006 [MUSS] - Fehler bei einer einzelnen Nachricht dürfen den gesamten Abruf nicht zwangsläufig abbrechen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Fehlerhafte Nachricht wird protokolliert; weitere Nachrichten werden geprüft.

LF-ABR-007 [MUSS] - Die Standardoption muss Nachrichten auf dem Server belassen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Nach erfolgreichem Abruf wird kein POP3-Löschkommando gesendet.

LF-ABR-008 [SOLL] - Eine spätere Löschoption darf nur explizit, kontoabhängig und mit deutlicher Warnung aktivierbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Ohne bewusste Aktivierung ist keine Serverlöschung möglich.

LF-ABR-009 [MUSS] - Der Abruf muss einen Fortschritt mit Anzahl, Datenmenge, Laufzeit und aktueller Phase anzeigen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Fortschritt wird während des Laufs aktualisiert und am Ende als Bericht gespeichert.

LF-ABR-010 [MUSS] - Die Anwendung muss zwischen normalem Abruf, vollständigem Abruf und lokalem Import unterscheiden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Die fachlichen Daten werden über FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: FetchRun, RemoteMessageIdentity, FetchItem, ProvenanceRecord. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Download Use Cases. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Provenienzfeld zeigt die Quelle jeder Nachricht.

23.3 Duplikaterkennung

LF-DUP-001 [MUSS] - Für jede vollständig geladene Rohmail muss ein SHA-256-Hash über die unveränderten Bytes berechnet werden.

Technische Realisierung	Der Hash wird mit IncrementalHash/SHA256 unmittelbar über die unveränderten empfangenen Bytes berechnet. Der Digest wird als 32-Byte-BLOB und zusätzlich für Berichte hexadezimal gespeichert. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Hash ist deterministisch und in der Datenbank gespeichert.

LF-DUP-002 [MUSS] - Die Duplikatprüfung muss mindestens Konto-ID, Rohdatenhash und Byte-Länge berücksichtigen.

Technische Realisierung	Ein UNIQUE-Index auf (account_id, raw_sha256, raw_length) verhindert parallele oder wiederholte Doppelübernahme auch dann, wenn zwei Abläufe nahezu gleichzeitig dieselbe Nachricht verarbeiten. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Zwei identische Mails im gleichen Konto werden nur einmal als kanonische Mail gespeichert.

LF-DUP-003 [MUSS] - Eine während „Get all“ als bekannt erkannte temporäre Kopie muss automatisch verworfen werden.

Technische Realisierung	Die Temp-Datei wird erst nach erfolgreicher Hashermittlung und Datenbankprüfung gelöscht. Das Deduplizierungsereignis referenziert die kanonische MessageId, UIDL, Hash, Länge und den Abrufverlauf. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Lokaler Bestand wächst bei erneutem vollständigem Abruf nicht durch exakte Duplikate.

LF-DUP-004 [MUSS] - Die vorhandene kanonische lokale Mail und die Servermail dürfen durch die Duplikatbereinigung nicht gelöscht werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Nur die neue temporäre Downloadkopie wird entfernt.

LF-DUP-005 [SOLL] - Der Anwender muss einen lokalen Bestandsprüflauf für bereits vorhandene Duplikate starten können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Lauf gruppiert exakte Duplikate und erzeugt vor Änderungen einen Bericht.

LF-DUP-006 [MUSS] - Automatische Bestandsbereinigung darf nur beweisbar identische Rohmails entfernen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Unterschiedliche Rohdatenhashes werden niemals automatisch zusammengeführt.

LF-DUP-007 [SOLL] - Message-ID, UIDL und semantische Merkmale sollen als Diagnoseinformationen gespeichert werden, dürfen den Rohdatenhash aber nicht ersetzen.

Technische Realisierung	Der schnelle Abruf liest die UIDL-Liste einmal pro Sitzung, gleicht sie über einen eindeutigen Index (AccountId, Uidl) ab und lädt nur unbekannte Einträge vollständig. UIDL wird niemals als globale, kontoübergreifende Identität behandelt. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Diagnose zeigt alle Kennungen; exakte Entscheidung basiert weiterhin auf Rohdaten.

LF-DUP-008 [MUSS] - Jeder Deduplizierungsvorgang muss nachvollziehbar protokolliert werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Die fachlichen Daten werden über MessageFingerprint, CanonicalMessage, DeduplicationEvent abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: MessageFingerprint, CanonicalMessage, DeduplicationEvent. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Deduplication + Persistence.Sqlite. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Bericht nennt kanonische Nachricht, verworfene temporäre Kopie, Hash und Zeitpunkt.

23.4 Speicherung und Provenienz

LF-SPR-001 [MUSS] - Rohmails müssen unverändert und getrennt von abgeleiteten Daten gespeichert werden.

Technische Realisierung	Rohmails werden als schreibgeschützte .eml-Dateien unter einem hashbasierten Verzeichnispfad abgelegt. Parser, Analyzer und Renderer erhalten ausschließlich lesenden Zugriff; abgeleitete Inhalte liegen in getrennten Tabellen. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Originaldatei bleibt bytgleich; Parser überschreibt sie nicht.

LF-SPR-002 [MUSS] - Metadaten, Status, Analyseergebnisse und Indizes müssen in SQLite gespeichert werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Anwendung kann nach Neustart alle Zustände rekonstruieren.

LF-SPR-003 [MUSS] - Dateiübernahme und Datenbankregistrierung müssen absturzsicher koordiniert werden.

Technische Realisierung	Die Übernahme verwendet ein Write-Temp-Fsync-Rename-Verfahren. Erst nach atomarem Rename wird die SQLite-Transaktion auf RawStored gesetzt; ein Startup-Reconciler erkennt unvollständige Gegenstücke und stellt einen definierten Zustand her. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Kein regulärer Abbruch erzeugt einen als vollständig markierten Datensatz ohne Rohdatei.

LF-SPR-004 [MUSS] - Temporäre Dateien müssen in einem separaten Bereich gespeichert und bereinigbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Verwaiste Temp-Dateien können beim Start erkannt und sicher entfernt werden.

LF-SPR-005 [MUSS] - Jede Mail muss eine interne, providerunabhängige ID erhalten.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Interne ID bleibt auch bei Reprocessing und geänderten externen Kennungen stabil.

LF-SPR-006 [MUSS] - Provenienz muss Konto, Abrufart, Abrufzeit, UIDL, Rohhash, Größe und Speicherort enthalten.

Technische Realisierung	Der schnelle Abruf liest die UIDL-Liste einmal pro Sitzung, gleicht sie über einen eindeutigen Index (AccountId, Uidl) ab und lädt nur unbekannte Einträge vollständig. UIDL wird niemals als globale, kontoübergreifende Identität behandelt. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Provenienz ist in der Historienansicht sichtbar und exportierbar.

LF-SPR-007 [MUSS] - Schemaänderungen müssen über versionierte Migrationen erfolgen.

Technische Realisierung	Migrationen sind fortlaufend nummerierte, idempotenzgeprüfte SQL-Skripte. Vor jeder produktiven Migration wird ein Backup erstellt; SchemaVersion und Ergebnis werden in schema_migrations protokolliert. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Update von der vorherigen unterstützten Version erhält bestehende Daten.

LF-SPR-008 [MUSS] - Die lokale Datenablage muss in dokumentierten, benutzerbezogenen Windows-Verzeichnissen erfolgen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Die fachlichen Daten werden über RawMessage, MessageRecord, StorageTransaction, MigrationHistory abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: RawMessage, MessageRecord, StorageTransaction, MigrationHistory. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Persistence.Sqlite + FileSystemRawMessageStore. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Verbindungsabbruch während des Streams; Programmabsturz zwischen Datei- und DB-Schritt; erneuter Lauf mit identischem Serverbestand.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Installations- und Datenpfade sind im Handbuch benannt und werden nicht im Programmverzeichnis vermischt.

23.5 MIME und sichere Darstellung

LF-DAR-001 [MUSS] - Die Anwendung muss RFC-822-/MIME-Nachrichten mit MimeKit oder gleichwertiger Bibliothek parsen.

Technische Realisierung	MimeKit übernimmt RFC-/MIME-Dekodierung. Der Adapter mappt MimeMessage und MimeEntity in eigene Domain-/DTO-Typen, sodass keine MimeKit-Typen die Mail-Schicht verlassen. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Plaintext, HTML, Multipart und Anhänge werden aus typischen Testmails korrekt erkannt.

LF-DAR-002 [MUSS] - Plaintext und HTML müssen als getrennte abgeleitete Ansichten verfügbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Nutzer kann bevorzugte Ansicht wählen; Rohmail bleibt unverändert.

LF-DAR-003 [MUSS] - Die Anwendung muss verschachtelte MIME-Strukturen, Base64 und quoted-printable verarbeiten.

Technische Realisierung	MimeKit übernimmt RFC-/MIME-Dekodierung. Der Adapter mappt MimeMessage und MimeEntity in eigene Domain-/DTO-Typen, sodass keine MimeKit-Typen die Mail-Schicht verlassen. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Referenzmails mit diesen Kodierungen werden lesbar angezeigt.

LF-DAR-004 [MUSS] - Externe Bilder und Ressourcen müssen standardmäßig blockiert sein.

Technische Realisierung	Der HTML-Sanitizer entfernt oder ersetzt externe src-, srcset-, CSS-url- und ähnlichen Ressourcenbezüge. Ein ResourceRequestInterceptor blockiert verbleibende Netzanfragen standardmäßig vollständig. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Beim Öffnen einer HTML-Mail erfolgt ohne Nutzerfreigabe kein externer Abruf.

LF-DAR-005 [MUSS] - Skripte, aktive Inhalte und gefährliche URL-Schemata dürfen nicht ausgeführt werden.

Technische Realisierung	Script-, iframe-, object-, embed-, form- und Meta-Refresh-Elemente werden vor dem Rendern entfernt. Navigation und neue Fenster werden im WebView-Host abgefangen; file:, javascript:, data: und benutzerdefinierte Schemata sind blockiert. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Testmails können kein JavaScript, file:, data:-Download oder benutzerdefiniertes Protokoll automatisch starten.

LF-DAR-006 [MUSS] - Die echte Linkadresse muss vor dem Öffnen sichtbar sein.

Technische Realisierung	Hover, Fokus und Bestätigungsdialog zeigen Original-URL, normalisierte URL, registrierbare Domain und IDN/Punycode. Das Öffnen erfolgt erst über einen UI-Service nach Abschluss der Warnbewertung. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Mouseover oder Kontextaktion zeigt normalisierte Ziel-URL und Domain.

LF-DAR-007 [MUSS] - HTML-Mail muss in einer isolierten, kontrollierten Rendering-Umgebung angezeigt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Navigation und Ressourcenanforderungen werden durch die Anwendung abgefangen.

LF-DAR-008 [MUSS] - Eine Rohdatenansicht muss vollständige Header, Body-Quelltext und MIME-Baum bereitstellen.

Technische Realisierung	MimeKit übernimmt RFC-/MIME-Dekodierung. Der Adapter mappt MimeMessage und MimeEntity in eigene Domain-/DTO-Typen, sodass keine MimeKit-Typen die Mail-Schicht verlassen. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Nachricht kann ohne externe Werkzeuge untersucht werden.

LF-DAR-009 [SOLL] - Der Anwender muss Schriftgröße, Zoom und bevorzugte Textansicht einstellen können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Die fachlichen Daten werden über ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ParsedMessage, MimePartDescriptor, RenderPolicy, LinkObservation. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Mail.MailKit + Security.Html + Presentation. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	defektes MIME; unbekannter Zeichensatz; HTML ohne Plaintext; verschachtelte multipart-Struktur.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Einstellungen bleiben nach Neustart erhalten.

23.6 Lokales Warnsystem

LF-WAR-001 [MUSS] - Die Anwendung muss alle anklickbaren Links, Button-Ziele, Bildlinks und Formularziele aus HTML-Mail lokal extrahieren.

Technische Realisierung	Ein HTML-LinkExtractor traversiert den sanitizten und den ursprünglichen DOM-Baum und erfasst a[href], area[href], form[action], button/formaction, bildbasierte Links sowie CSS-/Meta-Weiterleitungen als LinkObservation. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Analyse listet jedes gefundene Ziel mit Quelle und sichtbarem Text auf.

LF-WAR-002 [MUSS] - Sichtbarer Linktext und tatsächliches Ziel müssen auf Widersprüche geprüft werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Ein Linktext „sparkasse.de“ mit abweichendem href erzeugt einen erklärten Warnhinweis.

LF-WAR-003 [MUSS] - Zieldomains müssen normalisiert und sowohl als Unicode- als auch Punycode-Darstellung angezeigt werden.

Technische Realisierung	Domains werden mit IdnMapping in Unicode und ASCII/Punycode dargestellt. Gemischte Schriftsysteme, Confusable-Zeichen und Abweichungen zwischen Anzeige- und ASCII-Form erzeugen eigenständige Evidenz. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: IDN-Domain ist in beiden Formen sichtbar; verdächtige Zeichenmischung wird markiert.

LF-WAR-004 [MUSS] - URLs mit IP-Adresse statt Domain, Klartext-HTTP, ungewöhnlichen Ports oder gefährlichen Schemata müssen gesondert bewertet werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Jeder Indikator erscheint einzeln mit Begründung.

LF-WAR-005 [MUSS] - Die Linkdomain muss mit From-, Sender-, Reply-To-, Return-Path- und authentifizierten Domains verglichen werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Analyse zeigt Übereinstimmungen und Abweichungen, ohne daraus allein ein endgültiges Urteil abzuleiten.

LF-WAR-006 [MUSS] - Das System muss lokale Marken- und Vertrauensprofile mit zulässigen Domains, Unterdomains und Dienstleistern unterstützen.

Technische Realisierung	BrandTrustProfile enthält primäre Domains, erlaubte Subdomains, bekannte Versand-/Trackingdienstleister, Gültigkeitszeitraum, Herausgeber und Versionsnummer. Profile sind lokal editierbar und werden nie automatisch aus einer Mail erweitert. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Für ein Profil „Sparkasse“ können mehrere legitime Domains gepflegt und versioniert werden.

LF-WAR-007 [MUSS] - Abweichungen von einem aktiv passenden Markenprofil müssen deutlich, aber erklärbar gewarnt werden.

Technische Realisierung	BrandTrustProfile enthält primäre Domains, erlaubte Subdomains, bekannte Versand-/Trackingdienstleister, Gültigkeitszeitraum, Herausgeber und Versionsnummer. Profile sind lokal editierbar und werden nie automatisch aus einer Mail erweitert. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Warnung nennt erwartete und tatsächliche Domain sowie auslösende Regel.

LF-WAR-008 [MUSS] - Die Anwendung muss Authentication-Results für SPF, DKIM und DMARC auswerten und verständlich anzeigen, sofern vorhanden.

Technische Realisierung	Der Parser liest Authentication-Results als unvertrauenswürdige Beobachtung und ordnet SPF-, DKIM- und DMARC-Resultate samt authserv-id zu. Ein Ergebnis wird nur als Hinweis angezeigt, nicht als lokal kryptografisch neu geprüfter Beweis. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Pass/Fail/Neutral/None werden mit der betroffenen Domain dargestellt.

LF-WAR-009 [MUSS] - Reply-To-Abweichungen und Display-Name-Spoofing müssen als eigene Indikatoren dargestellt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Abweichung wird unabhängig von Linkanalyse sichtbar.

LF-WAR-010 [SOLL] - Externe Tracking-Ressourcen und sehr kleine unsichtbare Bilder sollen erkannt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Analyse listet externe Hosts, Abmessungen soweit bekannt und Trackingverdacht auf.

LF-WAR-011 [MUSS] - Dateiname, deklarierter MIME-Typ und erkannter Dateityp von Anhängen müssen auf Widersprüche geprüft werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Doppelendung oder Typabweichung erzeugt Warnhinweis.

LF-WAR-012 [MUSS] - Das Warnsystem muss mehrere Einzelindikatoren zu Warnstufen zusammenführen, aber alle Gründe einzeln anzeigen.

Technische Realisierung	WarningAggregator berechnet keine undurchsichtige Wahrscheinlichkeit, sondern ordnet versionierte Regelbefunde deterministisch einer Stufe (Info, Hinweis, Erhöht, Hoch) zu. Die UI zeigt alle beitragenden Befunde und ihre Evidenz. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Keine Warnstufe erscheint ohne nachvollziehbare Begründungsliste.

LF-WAR-013 [MUSS] - Das System darf niemals behaupten, eine Mail sei garantiert sicher.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Niedrige Warnstufe wird als „keine Auffälligkeit erkannt“ und nicht als Sicherheitsgarantie formuliert.

LF-WAR-014 [MUSS] - Links mit erhöhter Warnstufe dürfen erst nach bewusster Bestätigung geöffnet werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Direkter Klick zeigt Zwischendialog mit Ziel, Gründen und Abbruchoption.

LF-WAR-015 [SOLL] - Der Nutzer muss eine Domain für ein Markenprofil lokal vertrauen oder eine Fehlwarnung dokumentieren können.

Technische Realisierung	BrandTrustProfile enthält primäre Domains, erlaubte Subdomains, bekannte Versand-/Trackingdienstleister, Gültigkeitszeitraum, Herausgeber und Versionsnummer. Profile sind lokal editierbar und werden nie automatisch aus einer Mail erweitert. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Ausnahme ist sichtbar, widerrufbar und mit Zeitstempel versehen.

LF-WAR-016 [MUSS] - Online-Reputation oder automatisches Aufrufen von Links darf in V1 nicht ohne ausdrückliche Aktivierung erfolgen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Standardanalyse erzeugt keinen Netzwerkverkehr zu Mailzielen oder Reputationsdiensten.

LF-WAR-017 [MUSS] - Warnregeln und Analyseergebnisse müssen versioniert sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Historie zeigt Analyzer-ID, Version, Eingabehash und Ergebniszeitpunkt.

LF-WAR-018 [MUSS] - Das Warnsystem muss über stabile Analyzer-Schnittstellen erweiterbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Neue interne Analyzer können registriert werden, ohne die UI oder den Mailabruf zu ändern.

LF-WAR-019 [SOLL] - Der Anwender muss einen Sicherheitsbericht für eine Nachricht exportieren können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Bericht enthält Herkunft, Hash, Links, Indikatoren, Warnstufe, Methodik und Grenzen.

LF-WAR-020 [SOLL] - Redirector-, Kurzlink- und Trackingdomains sollen lokal anhand konfigurierbarer Listen erkannt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Bekannte Muster werden markiert; ohne Netzwerkzugriff wird das endgültige Redirectziel nicht behauptet.

23.7 Verfassen, Versand und Gesendet-Upload

LF-VSD-001 [MUSS] - Die Anwendung muss Nachrichten über SMTP mit TLS beziehungsweise STARTTLS versenden können.

Technische Realisierung	SendMessageUseCase erzeugt genau eine kanonische MimeMessage und speichert sie vor dem SMTP-Aufruf in der Outbox. Dieselben Bytes werden nach erfolgreichem Versand als lokale Gesendet-Rohmail und für IMAP APPEND verwendet. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Testversand mit GMX oder IONOS ist ohne unverschlüsselte Fallback-Verbindung möglich.

LF-VSD-002 [MUSS] - Neue Nachricht, Antworten, Allen antworten und Weiterleiten müssen unterstützt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Header und Threadreferenzen werden korrekt gesetzt.

LF-VSD-003 [MUSS] - Entwürfe müssen lokal automatisch gespeichert und nach Neustart wiederherstellbar sein.

Technische Realisierung	Entwürfe werden debounced und versioniert lokal gespeichert. Der Editor arbeitet mit einem Draft DTO; Anhänge werden bereits in einen verwalteten Entwurfsbereich kopiert, damit externe Dateien nicht unbemerkt verschwinden. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Unversendeter Text geht nach kontrolliertem Neustart nicht verloren.

LF-VSD-004 [MUSS] - To, Cc, Bcc, Reply-To, Text, HTML, Signatur und Anhänge müssen unterstützt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Versendete Referenzmail wird in anderem Standardclient korrekt dargestellt.

LF-VSD-005 [SOLL] - Vor Versand muss bei leerem Betreff und erwähntem, aber fehlendem Anhang gewarnt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Warnung kann bewusst bestätigt werden.

LF-VSD-006 [MUSS] - Der Versand muss über eine lokale Outbox mit eindeutigen Zuständen erfolgen.

Technische Realisierung	OutboxItem besitzt explizite Zustände Pending, Sending, Sent, SendUncertain, RetryRequired und FailedPermanent. Zustandswechsel erfolgen transaktional und werden in der Versandhistorie protokolliert. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Pending, Sending, Sent, SendUncertain, RetryRequired und Failed sind unterscheidbar.

LF-VSD-007 [MUSS] - Nach erfolgreichem SMTP-Versand muss die exakt versendete MIME-Nachricht lokal archiviert werden.

Technische Realisierung	SendMessageUseCase erzeugt genau eine kanonische MimeMessage und speichert sie vor dem SMTP-Aufruf in der Outbox. Dieselben Bytes werden nach erfolgreichem Versand als lokale Gesendet-Rohmail und für IMAP APPEND verwendet. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Lokale Gesendet-Mail besitzt denselben Message-ID- und Inhaltsstand wie der Versand.

LF-VSD-008 [MUSS] - Die gesendete Nachricht muss per IMAP APPEND in den serverseitigen Gesendet-Ordner hochgeladen werden können.

Technische Realisierung	SentMessageUploadService ermittelt den Zielordner, öffnet ihn schreibbar und verwendet MailKit AppendAsync mit der exakt versendeten MimeMessage und dem Flag Seen. APPENDUID wird, falls verfügbar, gespeichert. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Andere IMAP-Clients sehen die Nachricht nach erfolgreichem Upload.

LF-VSD-009 [MUSS] - Der Gesendet-Ordner muss über Special-Use, Providerprofil oder Nutzerauswahl bestimmbar sein.

Technische Realisierung	FolderResolver priorisiert IMAP Special-Use \Sent, danach das Providerprofil und zuletzt eine gespeicherte Nutzerauswahl. Ein automatisch neu angelegter Ordner erfolgt nur nach bewusster Bestätigung. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Kein fest verdrahteter Ordnername ist erforderlich.

LF-VSD-010 [MUSS] - Der IMAP-Upload muss idempotent und gegen unklare Netzwerkabbrüche abgesichert sein.

Technische Realisierung	Vor Retry eines unklaren APPEND wird im Zielordner begrenzt nach Message-ID, Datum, Größe und lokalem Hashhinweis gesucht. Erst wenn keine Übereinstimmung vorliegt, darf erneut angehängt werden. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Bei UploadUncertain wird vor Wiederholung nach Message-ID und gespeicherten Merkmalen geprüft.

LF-VSD-011 [MUSS] - Fehlgeschlagene Uploads müssen in einer sichtbaren Warteschlange erneut gestartet werden können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Nutzer sieht Status, letzten Fehler und Retry-Aktion.

LF-VSD-012 [MUSS] - SMTP-Erfolg und IMAP-Upload-Erfolg müssen getrennt dargestellt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Eine versendete, aber noch nicht hochgeladene Mail wird nicht als Versandfehler dargestellt.

LF-VSD-013 [SOLL] - Die Anwendung muss Offline-Verfassen und späteren Versand vorbereiten.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Die fachlichen Daten werden über Draft, OutboxItem, SentMessage, SentUploadJob abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Draft, OutboxItem, SentMessage, SentUploadJob. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Sending + Mail.MailKit. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	SMTP-Annahme unklar; IMAP APPEND erfolgreich aber Antwort verloren; Gesendet-Ordner nicht vorhanden; Offline-Zustand.
Verifikation	Integrationstest gegen kontrollierten Mailserver beziehungsweise Fake-Adapter. Fachlicher Nachweis: Nachricht kann in der Outbox verbleiben und manuell erneut versendet werden.

23.8 Organisation, Suche und Regeln

LF-ORG-001 [MUSS] - Die Anwendung muss lokale Systemordner für Posteingang, Gesendet, Entwürfe, Outbox, Archiv, Papierkorb und Quarantäne bereitstellen.

Technische Realisierung	Entwürfe werden debounced und versioniert lokal gespeichert. Der Editor arbeitet mit einem Draft DTO; Anhänge werden bereits in einen verwalteten Entwurfsbereich kopiert, damit externe Dateien nicht unbemerkt verschwinden. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Jede Nachricht ist mindestens einem nachvollziehbaren lokalen Bereich zugeordnet.

LF-ORG-002 [MUSS] - Nachrichten müssen gelesen/ungelesen, markiert und mit lokalen Tags versehen werden können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Status bleibt nach Neustart erhalten.

LF-ORG-003 [MUSS] - Lokale Ordner müssen angelegt, umbenannt und gelöscht werden können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Nachrichten können verlustfrei verschoben werden.

LF-ORG-004 [SOLL] - Eine einheitliche Ansicht über mehrere Konten muss verfügbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Unified Inbox zeigt Herkunftskonto eindeutig an.

LF-ORG-005 [MUSS] - Die Nachrichtenliste muss sortierbar, filterbar und für große Bestände virtuell beziehungsweise performant sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: 100.000 Metadatensätze lassen sich ohne vollständiges Laden aller Bodies durchsuchen und darstellen.

LF-ORG-006 [MUSS] - Eine lokale Volltextsuche über Betreff, Absender, Empfänger und normalisierten Text muss verfügbar sein.

Technische Realisierung	SQLite FTS5 indiziert Betreff, normalisierten Absender-/Empfängertext und bereinigten Plaintext. FTS-Einträge referenzieren MessageId und werden nach erfolgreichem Parsing in derselben logischen Verarbeitung aktualisiert. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Typische Suche liefert Treffer konto- und ordnerübergreifend.

LF-ORG-007 [MUSS] - Suche nach Datum, Status, Tag, Anhang und Dateiname muss möglich sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Filter lassen sich kombinieren und zurücksetzen.

LF-ORG-008 [SOLL] - Gespeicherte Suchen und virtuelle Ordner sollen vorbereitet werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Mindestens eine Suchdefinition kann gespeichert und erneut ausgeführt werden.

LF-ORG-009 [SOLL] - Einfache Regeln für Absender, Betreff, Header und Tags sollen auf neue und vorhandene Mails anwendbar sein.

Technische Realisierung	Regeln werden als deklarative Bedingungen und whitelist-basierte Aktionen gespeichert. Es gibt in V1 keine Shell-, PowerShell-, JavaScript- oder beliebige CLR-Ausführung. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Regeltest zeigt betroffene Nachrichten vor Ausführung.

LF-ORG-010 [MUSS] - Regelaktionen dürfen keine unbekannten externen Skripte ausführen.

Technische Realisierung	Script-, iframe-, object-, embed-, form- und Meta-Refresh-Elemente werden vor dem Rendern entfernt. Navigation und neue Fenster werden im WebView-Host abgefangen; file:, javascript:, data: und benutzerdefinierte Schemata sind blockiert. Primär betroffen sind Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Die fachlichen Daten werden über Folder, Tag, MessageTag, SearchDocument, RuleDefinition abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Folder, Tag, MessageTag, SearchDocument, RuleDefinition. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Application.Organization + Persistence.Sqlite/FTS5. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	100.000+ Nachrichten; Index noch nicht vollständig; gelöschter Tag/Ordner; Suchabbruch.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: V1 enthält nur klar definierte interne Aktionen.

23.9 Anhänge

LF-ANH-001 [MUSS] - Anhänge müssen mit Name, Größe, MIME-Typ und Hash angezeigt werden.

Technische Realisierung	AttachmentDescriptor speichert MIME-Pfad, Dateiname, Größe, Content-ID, deklarierten Typ, erkannten Signatortyp und Hash. Speichern erfolgt über einen sicheren Dateidialog mit Kollisionsstrategie und ohne automatisches Öffnen. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Die fachlichen Daten werden über Attachment, AttachmentFingerprint, AttachmentClassification abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Attachment, AttachmentFingerprint, AttachmentClassification. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Doppelendung; MIME-Typ stimmt nicht mit Signatur überein; sehr großer Anhang; Dateinamenskollision.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Alle Werte sind in Detailansicht und Export verfügbar.

LF-ANH-002 [MUSS] - Anhänge müssen einzeln und gesammelt in ein gewähltes Verzeichnis gespeichert werden können.

Technische Realisierung	AttachmentDescriptor speichert MIME-Pfad, Dateiname, Größe, Content-ID, deklarierten Typ, erkannten Signatortyp und Hash. Speichern erfolgt über einen sicheren Dateidialog mit Kollisionsstrategie und ohne automatisches Öffnen. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Die fachlichen Daten werden über Attachment, AttachmentFingerprint, AttachmentClassification abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Attachment, AttachmentFingerprint, AttachmentClassification. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Doppelendung; MIME-Typ stimmt nicht mit Signatur überein; sehr großer Anhang; Dateinamenskollision.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Dateinamen werden sicher behandelt; vorhandene Dateien nicht still überschrieben.

LF-ANH-003 [MUSS] - Ausführbare oder anderweitig riskante Dateitypen dürfen nicht direkt automatisch geöffnet werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Die fachlichen Daten werden über Attachment, AttachmentFingerprint, AttachmentClassification abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Attachment, AttachmentFingerprint, AttachmentClassification. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Doppelendung; MIME-Typ stimmt nicht mit Signatur überein; sehr großer Anhang; Dateinamenskollision.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Vor Öffnen erscheint deutliche Warnung oder Speichern-ist-erforderlich-Verhalten.

LF-ANH-004 [MUSS] - Die Anwendung muss SHA-256 für Anhänge berechnen; SHA-512 soll optional möglich sein.

Technische Realisierung	AttachmentDescriptor speichert MIME-Pfad, Dateiname, Größe, Content-ID, deklarierten Typ, erkannten Signaturtyp und Hash. Speichern erfolgt über einen sicheren Dateidialog mit Kollisionsstrategie und ohne automatisches Öffnen. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Die fachlichen Daten werden über Attachment, AttachmentFingerprint, AttachmentClassification abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Attachment, AttachmentFingerprint, AttachmentClassification. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Doppelendung; MIME-Typ stimmt nicht mit Signatur überein; sehr großer Anhang; Dateinamenskollision.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Hashwerte sind reproduzierbar und exportierbar.

LF-ANH-005 [MUSS] - Inline-Ressourcen müssen von normalen Anhängen unterscheidbar sein.

Technische Realisierung	AttachmentDescriptor speichert MIME-Pfad, Dateiname, Größe, Content-ID, deklarierten Typ, erkannten Signaturtyp und Hash. Speichern erfolgt über einen sicheren Dateidialog mit Kollisionsstrategie und ohne automatisches Öffnen. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Die fachlichen Daten werden über Attachment, AttachmentFingerprint, AttachmentClassification abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Attachment, AttachmentFingerprint, AttachmentClassification. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Doppelendung; MIME-Typ stimmt nicht mit Signatur überein; sehr großer Anhang; Dateinamenskollision.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: CID-Bilder werden korrekt der HTML-Mail zugeordnet.

LF-ANH-006 [SOLL] - Gängige technische Dateitypen sollen lokal klassifiziert werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Die fachlichen Daten werden über Attachment, AttachmentFingerprint, AttachmentClassification abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: Attachment, AttachmentFingerprint, AttachmentClassification. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Attachments + Raw Store. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Doppelendung; MIME-Typ stimmt nicht mit Signatur überein; sehr großer Anhang; Dateinamenskollision.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: PDF, Office, CSV, JSON, XML, Quellcode, Archiv, Bild und Logdatei werden unterschieden.

23.10 Scientific Foundation

LF-SCI-001 [MUSS] - Jede Nachricht muss einen „Message Laboratory“-Bereich mit Nachricht, Analyse, Rohdaten und Historie besitzen.

Technische Realisierung	MessageLaboratoryViewModel bündelt vier read-only Ansichten: Nachricht, Analyse, Rohdaten und Historie. Jede Ansicht lädt Daten asynchron über Application Queries und kennt keine Persistenzimplementierung. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Alle vier Sichten sind ohne externe Werkzeuge erreichbar.

LF-SCI-002 [MUSS] - Analyseläufe müssen Analyzer-ID, Version, Eingabehash, Parameterhash, Zeit, Status und Ergebnis speichern.

Technische Realisierung	AnalysisRun ist unveränderlich nach Abschluss. Ein erneuter Lauf erzeugt eine neue RunId; alte Resultate bleiben für Vergleich und Reproduzierbarkeit erhalten und können als Superseded markiert werden. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Ein Ergebnis ist später reproduzierbar zuordenbar.

LF-SCI-003 [MUSS] - Eine Analyse muss erneut ausgeführt werden können, ohne die Rohmail neu abzurufen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Reprocessing erzeugt neuen Lauf und erhält den alten Verlauf.

LF-SCI-004 [MUSS] - Die Anwendung muss eine technische MIME-Baumansicht erzeugen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Jeder Knoten zeigt Typ, Encoding, Größe und relevante Kennungen.

LF-SCI-005 [SOLL] - Received-Header müssen als sortierte Transportzeitleiste dargestellt werden können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Zeitpunkte, Relays und erkennbare Verzögerungen werden angezeigt.

LF-SCI-006 [SOLL] - Ausgewählte Nachrichten müssen als definierter Datensatz gespeichert werden können.

Technische Realisierung	DatasetDefinition speichert Auswahlkriterien, explizite MessageIds, Versionsnummer, Ersteller, Zeitpunkt und Datenstand. Exporte schreiben ein Manifest mit Inputhashes, Analyzer-Versionen und Pseudonymisierungsprofil. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Datensatz enthält Auswahlkriterien, Version, Anzahl, Zeitraum und Hashmanifest.

LF-SCI-007 [SOLL] - Datensätze und Analyseergebnisse sollen als CSV, JSON, JSONL, SQLite und Markdown exportierbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Export enthält Methodik, Schema- und Softwareversionen.

LF-SCI-008 [SOLL] - Pseudonymisierungsprofile für Adressen, IPs, Betreff, Body und Anhänge sollen unterstützt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Vorschau zeigt resultierende Daten vor Export.

LF-SCI-009 [SOLL] - Ein Untersuchungsbericht muss mindestens als Markdown oder HTML erzeugbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Bericht nennt Datenbasis, Methoden, Ergebnisse, Warnungen und Einschränkungen.

LF-SCI-010 [MUSS] - Das System muss zwischen Originaldaten, normalisierten Daten und Analyseergebnissen sichtbar unterscheiden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core.Scientific. Die fachlichen Daten werden über DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: DatasetDefinition, AnalysisRun, ReportArtifact, PseudonymizationProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Analysis.Core.Scientific. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Analyzer-Version geändert; Rohmailhash nicht mehr passend; unvollständige Analyse; Export mit Pseudonymisierung.
Verifikation	Fixture-basierter Test mit versionierten EML-Testmails. Fachlicher Nachweis: Keine abgeleitete Ansicht wird als Original bezeichnet.

23.11 Erweiterbarkeit

LF-EXT-001 [MUSS] - Kernprozessoren und Analyzer müssen über kleine, stabile Schnittstellen registriert werden.

Technische Realisierung	Kernmodule implementieren Verträge aus ExtensionModel und werden über Dependency Injection registriert. Der Host bestimmt Reihenfolge und Aktivierung; Application und Domain kennen keine konkreten Module. Primär betroffen sind Sasd.MailWorkbench.ExtensionModel. Die fachlichen Daten werden über ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion. Zuständiger Hauptbaustein: Sasd.MailWorkbench.ExtensionModel. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	nicht kompatible Contract-Version; Extension liefert ungültiges Ergebnis; Reihenfolgeabhängigkeit.
Verifikation	Architekturtest der Referenzrichtung. Fachlicher Nachweis: Neue interne Implementierung kann ohne Änderung des Abruf-Use-Cases ergänzt werden.

LF-EXT-002 [MUSS] - Erweiterungen müssen stabile IDs, Versionen und versionierte Ergebnisformate besitzen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.ExtensionModel. Die fachlichen Daten werden über ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion. Zuständiger Hauptbaustein: Sasd.MailWorkbench.ExtensionModel. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	nicht kompatible Contract-Version; Extension liefert ungültiges Ergebnis; Reihenfolgeabhängigkeit.
Verifikation	Architekturtest der Referenzrichtung. Fachlicher Nachweis: Ergebnis bleibt lesbar, wenn konkrete Implementierung nicht aktiv ist.

LF-EXT-003 [MUSS] - SQLite darf keine konkreten CLR-Typnamen als fachliche Kopplung speichern.

Technische Realisierung	Persistiert werden extension_id, extension_version, result_schema_version und result_json. AssemblyQualifiedName, Namespace oder konkrete Klassennamen sind nicht Teil des fachlichen Schemas. Primär betroffen sind Sasd.MailWorkbench.ExtensionModel. Die fachlichen Daten werden über ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion. Zuständiger Hauptbaustein: Sasd.MailWorkbench.ExtensionModel. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	nicht kompatible Contract-Version; Extension liefert ungültiges Ergebnis; Reihenfolgeabhängigkeit.
Verifikation	Architekturtest der Referenzrichtung. Fachlicher Nachweis: Gespeichert werden ID, Version, Schema-Version, Status und serialisierbares Ergebnis.

LF-EXT-004 [MUSS] - Version 1 muss kein dynamisches Laden fremder DLLs unterstützen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.ExtensionModel. Die fachlichen Daten werden über ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion. Zuständiger Hauptbaustein: Sasd.MailWorkbench.ExtensionModel. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	nicht kompatible Contract-Version; Extension liefert ungültiges Ergebnis; Reihenfolgeabhängigkeit.
Verifikation	Architekturtest der Referenzrichtung. Fachlicher Nachweis: Es existiert keine unkontrollierte Plugin-Ausführung im Hauptprozess.

LF-EXT-005 [MUSS] - Die Architektur muss spätere Erweiterungstypen für Analyser, Prozessoren, Exporter und Berichtsmodule vorbereiten.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.ExtensionModel. Die fachlichen Daten werden über ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion. Zuständiger Hauptbaustein: Sasd.MailWorkbench.ExtensionModel. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	nicht kompatible Contract-Version; Extension liefert ungültiges Ergebnis; Reihenfolgeabhängigkeit.
Verifikation	Architekturtest der Referenzrichtung. Fachlicher Nachweis: ExtensionModel-Projekt bleibt frei von GUI-, MailKit- und SQLite-Abhängigkeiten.

LF-EXT-006 [MUSS] - UI-Erweiterungen dürfen in V1 nicht Teil eines offenen Plugin-Vertrags sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.ExtensionModel. Die fachlichen Daten werden über ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: ExtensionDescriptor, ExtensionResultEnvelope, ContractVersion. Zuständiger Hauptbaustein: Sasd.MailWorkbench.ExtensionModel. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	nicht kompatible Contract-Version; Extension liefert ungültiges Ergebnis; Reihenfolgeabhängigkeit.
Verifikation	Architekturtest der Referenzrichtung. Fachlicher Nachweis: UI bleibt kontrolliert; spätere Panels werden separat geplant.

23.12 WinForms und Presentation Core

LF-UI-001 [MUSS] - Version 1 muss eine Windows-Forms-Oberfläche besitzen.

Technische Realisierung	Das ausführbare UI-Projekt targetet net10.0-windows und enthält nur Forms, UserControls, Ressourcen und UI-spezifische Services. Alle Use Cases und ViewModels werden per Konstruktor injiziert. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Anwendung lässt sich unter unterstütztem Windows als Desktopprogramm bedienen.

LF-UI-002 [MUSS] - Das WinForms-Projekt darf keine MailKit-, SQLite- oder fachliche Verarbeitung enthalten.

Technische Realisierung	Das ausführbare UI-Projekt targetet net10.0-windows und enthält nur Forms, UserControls, Ressourcen und UI-spezifische Services. Alle Use Cases und ViewModels werden per Konstruktor injiziert. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Architekturtests erkennen verbotene Referenzen.

LF-UI-003 [MUSS] - UI-unabhängige Zustände, ViewModels beziehungsweise Presenter müssen in einem separaten Presentation-Core-Projekt liegen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Kernpräsentationslogik lässt sich ohne WinForms instanziiieren und testen.

LF-UI-004 [MUSS] - Die Standardansicht muss Konto-/Ordernavigation, Nachrichtenliste und Lesebereich anbieten.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Drei Bereiche sind einzeln skalierbar und ihre Größe wird gespeichert.

LF-UI-005 [SOLL] - Profilbild, aktives Konto und Verbindungsstatus sollen kompakt im Kopfbereich sichtbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Anzeige benötigt keinen zusätzlichen Dialog.

LF-UI-006 [MUSS] - Die Oberfläche muss Normal-, Analyse- und Diagnose-Workspace unterscheiden können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Workspace-Wechsel ändert sichtbare Werkzeuge, nicht den fachlichen Datenbestand.

LF-UI-007 [SOLL] - Layout, Theme und Informationsdichte müssen getrennt konfigurierbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Mindestens Standard, dunkel und kompakt sind möglich, ohne Geschäftslogik zu ändern.

LF-UI-008 [MUSS] - Designer-Dateien dürfen keine handgeschriebene Geschäftslogik enthalten.

Technische Realisierung	*.Designer.cs wird ausschließlich durch den Visual-Studio-Designer erzeugt. Eigene Event- und Bindinglogik liegt in MainForm.cs beziehungsweise klar benannten Partial-Dateien; Architekturtests scannen Referenzen und Quelltextmuster. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Code Review und Architekturtest bestätigen die Trennung.

LF-UI-009 [MUSS] - Eventhandler müssen kurz bleiben und ausschließlich UI-unabhängige Dienste oder ViewModels aufrufen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewState-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: POP3-, Hash- oder SQL-Code ist in Forms nicht vorhanden.

LF-UI-010 [MUSS] - Die Oberfläche muss Tastaturbedienung, sichtbaren Fokus, skalierbare Schrift und High-DPI unterstützen.

Technische Realisierung	Tab-Reihenfolge, AccessibleName/Description, Shortcuts, AutoScaleMode.Dpi und LayoutPanels werden verbindlich geprüft. UI-Tests erfolgen mindestens bei 100 % und 150 % Skalierung. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewState-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Kernabläufe sind ohne Maus möglich; 150 % Skalierung bleibt nutzbar.

LF-UI-011 [MUSS] - Status und Warnungen dürfen nicht ausschließlich durch Farbe vermittelt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Text, Symbol oder Form ergänzt jede Farbcodierung.

LF-UI-012 [SOLL] - Frühere visuelle Studien müssen später als Themes oder Layoutvorlagen umsetzbar bleiben, ohne den Kern zu verändern.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Die fachlichen Daten werden über WorkspaceState, ViewModelState, UiPreference, LayoutProfile abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: WorkspaceState, ViewModelState, UiPreference, LayoutProfile. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Presentation.Core / Presentation.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	150 % DPI; Tastaturnavigation; abgebrochener asynchroner Vorgang; Form wird während Job geschlossen.
Verifikation	Unit-Test der Validierung und ViewModel-Logik; Architekturtest der Referenzrichtung. Fachlicher Nachweis: Themewechsel benötigt keine Änderung in Domain, Application oder Mail Engine.

23.13 Betrieb, Sicherheit und Diagnose

LF-BET-001 [MUSS] - Passwörter und Tokens müssen über einen Windows-geeigneten Secret Store geschützt werden.

Technische Realisierung	ISecretStore kapselt Windows Credential Manager oder DPAPI-geschützte Ablage. In SQLite steht nur eine SecretReference; Logs, Exceptions und Exporte maskieren Werte zentral über ISecretRedactor. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: SQLite und Konfigurationsdateien enthalten keine Klartextpasswörter.

LF-BET-002 [MUSS] - TLS-Zertifikate und Hostnamen müssen regulär geprüft werden; ungültige Zertifikate dürfen nicht still akzeptiert werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Verbindung schlägt mit verständlicher Diagnose fehl.

LF-BET-003 [MUSS] - Logs dürfen keine Passwörter, Tokens oder vollständige Mailbodies enthalten.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Automatisierter Logtest findet keine Testgeheimnisse oder Body-Testmarker.

LF-BET-004 [MUSS] - Technische Logs müssen strukturierte Korrelation über Konto-, Nachrichten-, Abruf- und Analyse-IDs ermöglichen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Ein Vorgang lässt sich über IDs vom Abruf bis zur Analyse verfolgen.

LF-BET-005 [SOLL] - Ein anonymisiertes Diagnosepaket muss exportierbar sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Vorschau zeigt enthaltene Dateien; Secrets und Mailinhalte sind standardmäßig ausgeschlossen.

LF-BET-006 [MUSS] - Die Anwendung muss ihre SQLite-Datenbank und Rohmails konsistent sichern können.

Technische Realisierung	BackupService erzeugt einen konsistenten SQLite-Snapshot, kopiert anschließend referenzierte Rohdateien und schreibt manifest.json mit Versionen, Dateilängen und SHA-256. Restore arbeitet nur in ein leeres oder explizit bestätigtes Profil. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Backup kann validiert und in leerem Profil wiederhergestellt werden.

LF-BET-007 [MUSS] - Einzelne Nachrichten müssen als EML exportiert und importiert werden können.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Exportierte Mail ist bytegleich oder dokumentiert normalisiert; Import verhindert exakte Duplikate.

LF-BET-008 [SOLL] - Kontoeinstellungen, Regeln, Markenprofile, Themes und Layouts müssen ohne Geheimnisse exportierbar sein.

Technische Realisierung	BrandTrustProfile enthält primäre Domains, erlaubte Subdomains, bekannte Versand-/Trackingdienstleister, Gültigkeitszeitraum, Herausgeber und Versionsnummer. Profile sind lokal editierbar und werden nie automatisch aus einer Mail erweitert. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Import in neues Profil rekonstruiert die Einstellungen.

LF-BET-009 [MUSS] - Die Anwendung muss eine Datenbankintegritätsprüfung und eine Wiederherstellungsdiagnose anbieten.

Technische Realisierung	MaintenanceUseCase führt PRAGMA quick_check/integrity_check, Fremdschlüsselprüfung, Rohdatei-Abgleich und Hash-Stichprobe aus. Reparaturen sind getrennte, bestätigungspflichtige Schritte und setzen ein Backup voraus. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Fehler werden verständlich gemeldet; keine automatische destruktive Reparatur ohne Backup.

LF-BET-010 [MUSS] - Alle sicherheitsrelevanten Standardwerte müssen datenschutzfreundlich sein.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Remote-Inhalte, Online-Reputation und Telemetrie sind standardmäßig aus.

LF-BET-011 [MUSS] - Telemetrie darf in Version 1 nicht vorausgesetzt werden.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Anwendung funktioniert vollständig ohne Herstellerkonto oder Cloud-Telemetrie.

LF-BET-012 [SOLL] - Die Anwendung muss ein nachvollziehbares Änderungs- und Migrationsprotokoll führen.

Technische Realisierung	Die Anforderung wird im zuständigen Use Case umgesetzt. Der Ablauf wird über eigene Verträge modelliert, persistente Zustände werden transaktional gespeichert und die Oberfläche erhält ausschließlich DTOs beziehungsweise ViewModels. Primär betroffen sind Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Die fachlichen Daten werden über BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration abgebildet.
Daten / Schnittstellen	Betroffene fachliche Daten: BackupManifest, DiagnosticPackage, AuditEvent, SchemaMigration. Zuständiger Hauptbaustein: Sasd.MailWorkbench.Security / Infrastructure / Host.WinForms. Öffentliche Übergaben erfolgen über versionierte Requests, Responses und DTOs.
Fehler- und Randfälle	Backup während laufender Jobs; beschädigte Datenbank; fehlende Rohdatei; Secret nicht verfügbar.
Verifikation	SQLite-/Dateisystem-Integrationstest in isoliertem Testprofil. Fachlicher Nachweis: Version, Schemaänderung, Zeitpunkt und Ergebnis sind lokal dokumentiert.

23.14 Nichtfunktionale Anforderungen

NF-ZUV-001 [MUSS] - Ein Absturz oder Verbindungsabbruch darf keine als vollständig markierte, aber unvollständige Nachricht erzeugen.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Application/Persistence/Mail zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-ZUV-002 [MUSS] - Wiederholte Abruf-, Import- und Uploadvorgänge müssen idempotent geplant sein.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Application/Persistence/Mail zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-ZUV-003 [MUSS] - Ein Fehler bei einer Nachricht darf andere unabhängige Nachrichten nicht unnötig blockieren.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Application/Persistence/Mail zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-PER-001 [MUSS] - Die Anwendung soll einen lokalen Bestand von mindestens 100.000 Nachrichten verwalten können.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Persistence/Presentation.Core zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-PER-002 [MUSS] - Nachrichtenlisten und Suchergebnisse müssen ohne vollständiges Laden aller Bodies reagieren.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Persistence/Presentation.Core zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.

NF-PER-003 [MUSS] - Hashberechnung und Download müssen streambasiert erfolgen, sodass große Nachrichten nicht vollständig im RAM liegen müssen.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Persistence/Presentation.Core zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-SEC-001 [MUSS] - Alle Netzverbindungen müssen moderne TLS-Verfahren und reguläre Zertifikatsprüfung verwenden.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Security/Mail/Presentation zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-SEC-002 [MUSS] - HTML-Inhalte sind grundsätzlich als nicht vertrauenswürdig zu behandeln.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Security/Mail/Presentation zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-SEC-003 [MUSS] - Sicherheitswarnungen müssen erklärbar, protokollierbar und ohne versteckte Cloudabhängigkeit sein.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Security/Mail/Presentation zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-DSG-001 [MUSS] - Mailinhalte und Analyseergebnisse bleiben standardmäßig lokal.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Security/Export zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-DSG-002 [SOLL] - Export- und Diagnosefunktionen müssen eine Vorschau und Pseudonymisierung ermöglichen.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Security/Export zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-BED-001 [MUSS] - Kernabläufe müssen per Tastatur bedienbar und bei 150 % Skalierung nutzbar sein.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Presentation.Core/WinForms zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-BED-002 [MUSS] - Fehlermeldungen müssen Handlungsoptionen nennen und dürfen nicht nur technische Ausnahmebezeichnungen zeigen.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Presentation.Core/WinForms zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-WAR-001 [MUSS] - Öffentliche Typen und Methoden müssen verständliche XML-Dokumentationskommentare erhalten.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich gesamte Solution zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer; Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	Fixture-basierter Test mit versionierten EML-Testmails; messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-WAR-002 [MUSS] - Komplexe Entscheidungen müssen das Warum im Code oder in ADRs dokumentieren.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind Sasd.MailWorkbench.Analysis.Core. Die fachlichen Daten werden über AnalysisRun, WarningFinding, BrandTrustProfile, AnalyzerDefinition abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich gesamte Solution zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	legitimer externer Dienstleister; fehlender Authentication-Results-Header; IDN mit gemischten Schriftsystemen; Linkverkürzer; Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	Fixture-basierter Test mit versionierten EML-Testmails; messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-TST-001 [MUSS] - Domain-, Application-, Persistence-, Mail-, Analysis- und Architekturregeln müssen automatisiert testbar sein.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich alle Testprojekte zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-TST-002 [MUSS] - Protokolltests gegen kontrollierte Testserver dürfen keine produktiven Postfächer voraussetzen.

Umsetzung	Diese Qualitätsanforderung wird als automatisiertes Quality Gate, Architekturregel oder messbares Akzeptanzkriterium umgesetzt und ist Bestandteil der Definition of Done für jedes betroffene Arbeitspaket. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich alle Testprojekte zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-PORT-001 [MUSS] - Domain, Application, Mail Engine, Persistence, Security, Extension Model und Presentation Core dürfen keine WinForms-Abhängigkeit besitzen.

Umsetzung	Das ausführbare UI-Projekt targetet net10.0-windows und enthält nur Forms, UserControls, Ressourcen und UI-spezifische Services. Alle Use Cases und ViewModels werden per Konstruktor injiziert. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich ArchitectureTests zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

NF-INT-001 [MUSS] - Standardformate EML, MIME, POP3, SMTP, IMAP APPEND und SQLite müssen interoperabel genutzt werden.

Umsetzung	SentMessageUploadService ermittelt den Zielordner, öffnet ihn schreibbar und verwendet MailKit AppendAsync mit der exakt versendeten MimeMessage und dem Flag Seen. APPENDUID wird, falls verfügbar, gespeichert. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich Mail/Persistence zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.

NF-DOC-001 [MUSS] - Installation, Entwicklung, Betrieb, Backup, Wiederherstellung und Tests müssen dokumentiert sein.

Umsetzung	BackupService erzeugt einen konsistenten SQLite-Snapshot, kopiert anschließend referenzierte Rohdateien und schreibt manifest.json mit Versionen, Dateilängen und SHA-256. Restore arbeitet nur in ein leeres oder explizit bestätigtes Profil. Primär betroffen sind mehrere Kernprojekte. Die fachlichen Daten werden über versionierte Domain- und Persistenzmodelle abgebildet.
Messgröße / Gate	Die Anforderung wird dem Bereich docs und Releaseprozess zugeordnet. Sie gilt als nicht erfüllt, wenn kein automatisierter Nachweis oder keine freigegebene manuelle Prüfanweisung existiert.
Randfälle	Messbarkeit im CI; Regression durch Abhängigkeitsupdate.
Nachweis	messbares Quality Gate in CI und dokumentierter manueller Nachweis

Anhang A - Datenbankschema

Die folgenden Tabellen bilden das fachliche Zielschema. Spaltennamen sind als technischer Entwurf verbindlich; Feinanpassungen benötigen eine Migration und Aktualisierung dieses Dokuments.

Tabelle	Zielspalten
mail_accounts	account_id TEXT PK; display_name; email_address; is_enabled; provider_key; pop_host/port/security; smtp_host/port/security; imap_host/port/security; secret_ref; created_at; updated_at
mail_identities	identity_id TEXT PK; account_id FK; display_name; email_address; reply_to; signature_text; signature_html; is_default
fetch_runs	run_id TEXT PK; account_id FK; mode; state; started_at; finished_at; server_count; checked_count; new_count; duplicate_count; failed_count; bytes_received
remote_message_identities	account_id FK; uidl; last_seen_at; message_id FK NULL; size; PK(account_id, uidl)
messages	message_id TEXT PK; account_id FK; state; subject_summary; sent_at; received_at; has_attachments; warning_level; created_at; updated_at
raw_messages	message_id PK/FK; raw_sha256 BLOB(32); raw_length INTEGER; storage_path; acquisition_kind; acquired_at; uidl; UNIQUE(account_id via message, hash, length)
parsed_messages	message_id FK; parser_id; parser_version; parsed_at; message_id_header; text_body; html_body_sanitized; normalized_text; parse_status; warnings_json; PK(message_id, parser_version)
message_addresses	id INTEGER PK; message_id FK; role; display_name; address_original; address_normalized; domain_ascii; domain_unicode

Tabelle	Zielspalten
attachments	attachment_id TEXT PK; message_id FK; mime_path; file_name; declared_mime; detected_type; content_id; size; sha256; sha512 NULL; is_inline; risk_class
outbox_items	outbox_id TEXT PK; account_id FK; mime_path; message_id_header; state; attempt_count; next_attempt_at; last_error_code; created_at; updated_at
sent_upload_jobs	upload_id TEXT PK; outbox_id FK; account_id FK; target_folder; state; imap_uid NULL; attempt_count; last_error_code; created_at; updated_at
analysis_runs	analysis_run_id TEXT PK; message_id FK; analyzer_id; analyzer_version; result_schema_version; input_hash; config_hash; state; result_json; started_at; completed_at
warning_findings	finding_id TEXT PK; analysis_run_id FK; rule_id; rule_version; severity; title; explanation; evidence_json; recommendation
brand_trust_profiles	profile_id TEXT; version INTEGER; name; primary_domains_json; allowed_domains_json; provider_domains_json; valid_from; valid_to; source; change_reason; PK(profile_id, version)
folders	folder_id TEXT PK; account_id NULL; parent_id NULL; kind; name; sort_order; is_system
message_folders	message_id FK; folder_id FK; added_at; PK(message_id, folder_id)
tags	tag_id TEXT PK; name UNIQUE; color NULL; icon NULL
message_tags	message_id FK; tag_id FK; PK(message_id, tag_id)
rules	rule_id TEXT PK; name; enabled; priority; condition_json; action_json; version; updated_at
rule_runs	rule_run_id TEXT PK; rule_id FK; message_id FK; rule_version; matched; actions_json; executed_at
schema_migrations	version INTEGER PK; name; checksum; applied_at; success; details
audit_events	event_id TEXT PK; occurred_at; event_type; correlation_id; account_id NULL; message_id NULL; details_json

```

PRAGMA foreign_keys = ON;
PRAGMA journal_mode = WAL;
PRAGMA synchronous = FULL;
PRAGMA busy_timeout = 5000;

CREATE UNIQUE INDEX ux_raw_exact
ON raw_messages(account_id, raw_sha256, raw_length);

CREATE VIRTUAL TABLE message_search_fts USING fts5(
  message_id UNINDEXED,
  subject,
  addresses,
  normalized_text,
  attachment_names,
  tokenize = 'unicode61 remove_diacritics 2'
);

```

Anhang B - Schnittstellenkatalog

Interface	Vertrag
IPop3MailboxClient	Connect/Test/List/GetMessageStream/Disconnect; keine Domainpersistenz.
ISmtpTransport	SendAsync einer vorbereiteten MIME-Datei; liefert serverseitige Antwort und Uncertainty.
IImapSentFolderClient	ResolveSentFolder, SearchCandidate, AppendAsync; nur Gesendet-Use-Case.
IMimeMessageParser	RawMessageInput → ParsedMessageResult mit Warnungen.
IRawMessageStore	CreateTemp, CommitCanonical, OpenRead, DeleteTemp, Verify.
IMessageRepository	Nachrichtenstatus, Summaries, Provenienz und Queries.
IFetchRunRepository	Abrufplan und Itemfortschritt.
IDeduplicationService	LookupExact, RegisterCanonical, RecordDuplicate.
IMessageFingerprintService	streambasierter SHA-256/optional SHA-512.
ISecretStore	Save/Get/Delete über SecretReference.
IHtmlSanitizer	Untrusted HTML → SanitizedHtml + observations.
ILinkObservationExtractor	HTML → LinkObservation-Liste.
IMessageAnalyzer	versionierter AnalysisInput → AnalysisResultEnvelope.
IWarningAggregator	Findings → WarningSummary.
IBrandTrustProfileRepository	Versionen laden/speichern und aktives Profil bestimmen.
ISearchIndex	Index/Remove/Search/Rebuild.
IBackupService	Create/Validate/Restore über Manifest.
IDiagnosticPackageService	Preview/Create mit Redaktionsprofil.
IUserNotificationService	UI-neutrale Information/Fehler/Bestätigung.
IFileDialogService	UI-neutrale Datei-/Ordnerauswahl.
ILinkOpenService	prüft WarningSummary und öffnet über Betriebssystem nach Bestätigung.

Anhang C - Glossar und technische Referenzen

Begriff	Bedeutung
APPENDUID	Vom IMAP-Server gelieferte UID der angehängten Nachricht, sofern UIDPLUS unterstützt wird.

Begriff	Bedeutung
Canonical Message	Die eine lokal gültige Originalinstanz einer exakten Rohmail pro Konto.
Get all	Vollständiger POP3-Abruf aller noch sichtbaren Servernachrichten mit Hashvergleich.
Idempotenz	Wiederholung eines Vorgangs erzeugt keinen zusätzlichen fachlichen Effekt.
Message Laboratory	Ansicht für Nachricht, Analyse, Rohdaten und Historie.
Provenienz	Nachweis von Quelle, Abrufart, Zeitpunkt, Hash, UIDL und Verarbeitung.
Raw Message	Unveränderte RFC-822-/MIME-Bytes der Mail.
SendUncertain	SMTP-Verbindung brach in einer Phase ab, in der die Serverannahme nicht sicher bekannt ist.
UploadUncertain	IMAP-APPEND könnte erfolgreich gewesen sein, Antwort ging aber verloren.
WarningFinding	Erklärbarer Einzelbefund mit Regelversion und Evidenz.

Technische Referenzen

- [R1] Microsoft: .NET and .NET Core official support policy - <https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core>
- [R2] Microsoft: Windows Forms overview / What is new in .NET 10 - <https://learn.microsoft.com/dotnet/desktop/winforms/>
- [R3] MailKit/MimeKit API documentation - <https://mimekit.net/docs/html/>
- [R4] Microsoft.Data.Sqlite overview - <https://learn.microsoft.com/dotnet/standard/data/sqlite/>
- [R5] RFC 1939 - Post Office Protocol Version 3.
- [R6] RFC 5321 - Simple Mail Transfer Protocol.
- [R7] RFC 9051 - Internet Message Access Protocol Version 4rev2.
- [R8] RFC 2045-2049 - Multipurpose Internet Mail Extensions.
- [R9] RFC 8601 - Message Header Field for Indicating Message Authentication Status.
- [R10] OWASP guidance for untrusted HTML, URL and file handling.

Anhang D - Freigabe

Freigabewirkung

Mit der Freigabe wird dieses Pflichtenheft zur technischen Grundlage für Implementierung und Abnahme von SASD Mail Workbench 1.0. Änderungen an MUSS-Anforderungen, Sicherheitsgrenzen, Persistenz- oder Protokollverhalten benötigen eine dokumentierte Änderungsentscheidung.

Rolle	Name / Organisation	Datum	Freigabe / Bemerkung
Product Owner	SASD-GmbH		
Technische Architektur			
Sicherheitsprüfung			
Test / Qualität			
Dokumentation			

Vollständigkeitsnachweis: 131 funktionale Anforderungen, 20 nichtfunktionale Anforderungen und 17 Abnahmeszenarien wurden aus dem Lastenheft eingelesen und in Kapitel 20 beziehungsweise 23 einzeln behandelt.

24. Änderungsblatt Version 1.1 und Stabilisierungsaufgaben

Dieses Kapitel präzisiert die technische Reihenfolge nach dem Qualitätsreview. Fachliche Muss-Anforderungen werden nicht reduziert.

24.1 Runtime und IDE

- 0.3.1 und 0.4.0 verwenden .NET 8, C# 12 und Visual Studio 2022.
- Migration auf .NET 10 LTS spätestens in 0.5.0.
- Bei Erreichen des .NET-8-Supportendes wird die Migration vorgezogen.
- Version 1.0 wird nur auf einer unterstützten LTS-Runtime veröffentlicht.
- Vor der Migration keine .NET-10-/C#-14-exklusiven Kernfunktionen.

24.2 Milestone 0.3.1

- SDK-Pinning, zentrale Paketverwaltung und Lockfiles.
- NUnit, dotnet test und Visual Studio Test Explorer.
- Stream-/Byteverträge und SHA-256 über Originalbytes.
- Import-Zustandsmaschine und Startup-Recovery.
- Versionierte SQLite-Migrationen und relative Speicherpfade.
- Stabile Processor-/Analyzer-IDs und Ergebnisversionen.
- Getrennte Abrufzeitstempel und eigener Cancelled-Status.

24.3 Zusätzliche technische Verpflichtungen

ID	Verpflichtung	Prüfnachweis
PH-STB-001	Rohmails bytengenau speichern.	Binär-, Encoding- und Zeilenendentests.
PH-STB-002	Hash und Länge streambasiert ermitteln.	Streaming- und Cancellationtests.
PH-STB-003	Get all nach Konto, Hash und Länge deduplizieren.	Wiederholter Get-all-Lauf.
PH-STB-004	Unvollständige Verarbeitung zustandsabhängig fortsetzen.	Retrytests.
PH-STB-005	Datei und SQLite über Staging und Recovery koordinieren.	Failure-Injection.
PH-STB-006	SQLite über versionierte Migrationen ändern.	Migrationstests.
PH-STB-007	Tests über dotnet test und Test Explorer ausführen.	Clean-Machine-TRX.
PH-STB-008	Restore durch SDK-Pinning und Lockfiles reproduzieren.	Locked Restore.
PH-STB-009	Stabile Processing-IDs und Versionen speichern.	Persistenz-/Reprocessingtest.

ID	Verpflichtung	Prüfnachweis
PH-STB-010	Cancellation als eigenen Status behandeln.	Abbruchtests.

24.4 Berichtigung

Die Anforderungszahlen sind korrekt: 131 funktionale, 20 nichtfunktionale Anforderungen und 17 Abnahmeszenarien. Die frühere gegenteilige Aussage war ein Prüfungsfehler.

24.5 Freigabe-Gates

- 0.4.0 beginnt erst nach erfolgreichem Clean-Machine-Restore, Build und Test von 0.3.1.
- Kritische Datenintegritätsbefunde werden nicht stillschweigend verschoben.
- Die Runtime-Migration erfolgt vor dem Produktrelease und spätestens bei Erreichen des .NET-8-Supportendes.
- Version 1.0 wird nicht auf einer nicht unterstützten Runtime freigegeben.